Deggendorf Institute of Technology

Master's Thesis

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Sooraj Sudhakaran

2026

TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham

Faculty of Applied Natural Sciences and Industrial Engineering

Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

master's thesis to obtain the academic degree:

Master of Engineering (M.Eng.)

submitted by: Sooraj Sudhakaran, Cham, XXXXXXXX

first examiner: Prof. Dr. Matthias Górka

supervisor: Martin Naumann

Cham, *April 21, 2026*

Declaration of Original Authorship

Name of student: Sooraj Sudhakaran
Name of first examiner: Prof. Dr. Matthias Górka
If external: Name of supervisor: Martin Naumann
Title of the master's thesis:
Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse
Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

1. I hereby declare that this final thesis is my own work, has not been submitted for any other degree at any other university or institution, does not contain or use any sources or resources other than those referenced, and that all direct and paraphrased quotes have been duly cited as such.

Cham, April 21, 2026 ....
Signature of student

2. I hereby consent to my completed final thesis being made available to a broader public through the library of Deggendorf Institute of Technology.

☐ yes ☐ no

If Yes:
I certify and warrant that I am the sole owner and holder of all rights to the final thesis, including the right to use the tables, figures, illustrations, plans, or similar, submitted therein and that no third-party rights or claims, nor any statutory provisions, will be infringed as a result of its publication.

Cham, April 21, 2026 ....
Signature of student

To be filled in by the first examiner if the author agrees with public accessibility of the master's thesis

Collection of a copy of the final thesis by the library, including the lending of the copy, is hereby endorsed / not endorsed.

☐ endorsed ☐ not endorsed

Cham, April 21, 2026 ....
Signature of first examiner

Declaration on the Use of AI Tools

During the preparation of this thesis, AI-based tools were used for language editing, grammar checking, formatting support, and assistance with LaTeX document preparation. The technical content, interpretations, results, and final responsibility for the thesis remain with the author.

Acknowledgements

Sincere gratitude is extended to the university supervisor, Gorka, for valuable guidance, continuous support, and constructive feedback throughout the development of this thesis.
Deep appreciation is also extended to the company supervisor, Martin, for practical insights, encouragement, and support during the work at KEBA Group. Further thanks are extended to Saif for support and assistance throughout the course of this work.
Gratitude is extended to KEBA Group for providing the necessary equipment and the opportunity to carry out this thesis within the company. The cooperation and support of the KEBA team members are also gratefully acknowledged.
The assistance of the Drag and Bot team with software simulation and robotic hardware simulation is gratefully acknowledged, as it significantly contributed to the successful completion of this work.
Sincere gratitude is also extended to Deggendorf Institute of Technology for providing the opportunity to pursue and complete this thesis.
Thanks are extended to the friends who helped provide access to important research papers through the University of Stuttgart.
Finally, deep appreciation is extended to family, friends, and colleagues for moral support and encouragement throughout the project.

Abstract

Press brake sheet metal bending is an important manufacturing process, but robotic implementations often still require manual programming. Operators define motion sequences, teach waypoints, and adapt programs for each new workpiece, which increases effort and limits flexibility.

This thesis presents an automated planning system for robotic press brake bending. The approach combines workpiece orientation determination, constraint-based grip planning, automatic regrip handling, and trajectory generation in a modular pipeline driven by geometric input and configuration data. The system generates robot programs without manual waypoint definition and includes validation steps for safe and feasible execution.

Evaluation in a virtual simulation environment covers varying workpiece dimensions, arbitrary initial orientations, regripping, and opposite-face bending. The results show executable robot programs and reduced programming effort compared with manual programming, supporting adaptable press brake automation for flexible manufacturing.

Keywords: robotic press brake bending; automatic robot program generation; grip planning; regrip planning; trajectory generation; flexible manufacturing.

Contents

Acknowledgements iii

Abstract iv

List of Figures ix

List of Tables x

List of Code Snippets xi

List of Abbreviations xii

List of Symbols xiii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Objectives . . . 2
- 1.4 Scope of Work . . . 2
- 1.5 Methodology Overview . . . 3
- 1.6 Thesis Structure . . . 3

2 State of the Art and Literature Review 5
- 2.1 Fundamentals of Industrial Robotics . . . 5
- 2.2 Sheet Metal Bending and Press Brake Systems . . . 5
  - 2.2.1 Process Characteristics and Automation Challenges . . . 6
- 2.3 Manual and Teach Pendant Programming . . . 7
  - 2.3.1 Limitations of Manual Programming . . . 7
  - 2.3.2 Challenges in Complex Manipulation Tasks . . . 8
  - 2.3.3 Transition Towards Automated Programming . . . 8
- 2.4 Offline Programming Approaches for Industrial Robots . . . 8
  - 2.4.1 Digital Twin and Simulation-Based Programming . . . 8
  - 2.4.2 Limitations of Offline Programming . . . 9
- 2.5 Automatic Robot Program Generation and Rule-Based Approaches . . . 10
  - 2.5.1 Task-Level Programming and Abstraction . . . 10
  - 2.5.2 Rule-Based and Constraint-Based Planning . . . 10
- 2.6 Robotic Sheet Metal Bending Systems . . . 11
  - 2.6.1 Regrip Planning in Sheet Metal Bending . . . 11
  - 2.6.2 Challenges of Regrip Integration in Industrial Systems . . . 12
  - 2.6.3 Grasp Planning for Sheet Metal Handling . . . 12

2.6.4 Trajectory Planning in Robotic Bending Systems 12
2.6.5 Limitations of Existing Robotic Bending Systems 13
2.7 Industrial Software Platforms and System Implementations 13
2.7.1 Simulation and Offline Programming Environments 13
2.7.2 Graphical Programming and Integrated Execution Systems . . . 14
2.7.3 Relevance to This Thesis 14
2.8 Research Gap and Motivation 14

3 System Concept and Methodology 16
3.1 Problem Definition and Objectives 16
3.2 Requirement Definition and Management 17
3.2.1 Functional Requirements 17
3.2.2 Non-Functional Requirements 18
3.2.3 System Constraints and Assumptions 21
3.2.4 Software Platform and Tool Requirements 22
3.2.4.1 SR-01 Virtual Execution Environment 22
3.2.4.2 SR-02 Communication Interface Requirement 23
3.2.4.3 Tool Centre Point Definition Requirement SR-03 23
3.2.5 System Traceability 24

4 Development of Concepts 27
4.1 Purpose and Scope of Concept Development 27
4.2 Concept A – Structured Operator Workflow with Guided Teach-In Programming 27
4.2.1 Methodological Framework 27
4.2.2 System Structure 29
4.2.3 Critical Evaluation 29
4.3 Concept B – Automatic Program Generation Based on Rules with Integrated Constraint Validation 30
4.3.1 Methodological Framework 30
4.3.2 System Structure 31
4.3.3 Critical Evaluation 32
4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation 33
4.4.1 Methodological Framework 33
4.4.2 System Structure 34
4.4.3 Critical Evaluation 34
4.5 Structural Comparison of Concepts 34
4.6 Concept Decision Using the Weighted Sum Model 36
4.6.1 Methodology 36
4.6.2 Evaluation Criteria and Weights 37
4.7 Result of the Weighted Sum Model 38
4.8 Analysis of Results and Concept Decision 40

5 Implementation **41**
5.1 Introduction and Scope . . . 41
5.2 System Architecture . . . 43
5.2.1 Layered Architecture Design and Data Flow . . . 43
5.2.2 Module Overview and Dependency Discipline . . . 45
5.2.3 Communication Stack and Authentication Interface Specification 47
5.2.4 Configuration Architecture and Data File Structure . . . 50
5.2.5 Operator Interface and Interactive Three-Dimensional Sheet Bending Viewer . . . 51
5.3 Development and Simulation Environment . . . 55
5.3.1 Simulation Platform, Robot Model, and Tool Centre Point Configuration . . . 55
5.3.2 Virtual Cell Layout and Spatial Zone Classification . . . 56
5.4 Pre-Execution Safety Validation . . . 58
5.4.1 Fixture Pose Deviation Validation . . . 58
5.4.2 Operator Authorization and Runtime Error Monitoring . . . 59
5.5 Workpiece Orientation Strategy . . . 59
5.5.1 Zone-Based Rotation Admissibility . . . 60
5.5.2 Pickup Orientation Validation . . . 60
5.5.3 Three-Level Orientation Selection Strategy . . . 60
5.6 Workpiece Handling and Gripper Control . . . 63
5.6.1 Vacuum Gripper Digital Input and Output Sequence . . . 63
5.6.2 Kinematic Object Attachment and Detachment . . . 63
5.7 Grip Position Planning and Feasibility Validation . . . 64
5.7.1 Sheet Geometry and Hole Detection . . . 64
5.7.2 Vacuum Holding Force Validation . . . 65
5.7.3 Grip Candidate Search and Selection . . . 65
5.8 Regrip Planning and Execution . . . 68
5.8.1 Regrip Decision Strategy . . . 69
5.8.2 Regrip Execution Sequence . . . 69
5.8.3 System Integration and Configuration . . . 70
5.9 Opposite-Face Bending Workflow . . . 71
5.9.1 Pre-Flip Verification and Transition Correction . . . 71
5.9.2 Sheet Flip Sequence and Opposite-Face Bending Execution . . . 71
5.10 Trajectory Generation . . . 72
5.10.1 Three-Phase Linear Trajectory Structure . . . 72
5.10.2 Waypoint Generation and Interpolation . . . 74
5.10.3 Kinematic Mapping and Joint Execution . . . 75
5.10.4 Workspace Constraints and Singularity Handling . . . 75
5.10.5 Motion Execution and Path Reduction . . . 75
5.11 System Reliability and Failure Mode Analysis . . . 75
5.11.1 Identification of Failure Modes . . . 76

5.11.2 Mitigation Strategies . . . 78
5.11.3 Discussion . . . 78
5.11.4 Conclusion . . . 78
5.12 Requirement Compliance Matrix . . . 79

6 Experimental Evaluation and Validation 83
6.1 Experimental Setup . . . 83
6.1.1 Test Environment . . . 83
6.1.2 Test Scenarios . . . 83
6.1.3 Evaluation Metrics . . . 84
6.2 Functional Validation . . . 84
6.2.1 Orientation Strategy Validation . . . 84
6.2.2 Grip Planning Validation . . . 85
6.2.3 Regrip Planning Validation . . . 85
6.2.4 Opposite-Face Bending Validation . . . 85
6.3 System-Level Performance Evaluation . . . 85
6.3.1 End-to-End Execution Results . . . 86
6.3.2 Execution Time Analysis . . . 86
6.3.3 Trajectory Quality and Feasibility . . . 86
6.3.4 Comparison with Manual Programming . . . 86
6.4 Robustness and Failure Analysis . . . 87
6.4.1 Failure Case Evaluation . . . 88
6.4.2 System Response to Errors . . . 88
6.4.3 Limitations of the System . . . 88
6.5 Discussion . . . 88

7 Conclusion and Future Work 90
7.1 Conclusion . . . 90
7.2 Contributions . . . 91
7.3 Limitations . . . 91
7.4 Future Work . . . 91
7.5 Final Remarks . . . 93

References 94

List of Figures

2.1 Robotic press brake bending system with integrated robot handling (Source: [10]). 6
2.2 Teach pendant interface for manual robot programming (Source: own work). 7
2.3 Offline robot programming environment with CAD-based simulation (Source: [12]). 9
2.4 Robotic sheet metal bending system with integrated robot and press brake (Source: [4]). 11

3.1 Tool Center Point parameters in drag&bot. 24

4.1 Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control. 28
4.2 Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control. 28
4.3 Virtual robotic environment used to evaluate the proposed concept (drag&bot platform). 33

5.1 Overall system workflow and execution pipeline. 43
5.2 Global system architecture for validation workflow. 45
5.3 Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [19]). 49
5.4 Robot control panel interface used for system operation and monitoring. 50
5.5 Operator workflow illustrating the sequential execution stages. Each stage must be completed before the next begins. All operator interactions occur within the drag&bot environment. *Source: own work.* . . . 52
5.6 Sheet metal workpiece model used for geometry analysis. Internal cutouts are detected and excluded from the grip candidate search region. Source: own work. 53
5.7 Operator workflow 54
5.8 Tool Centre Point (TCP) configuration of the UR5e robot used for motion planning. *Source: own work.* 56
5.9 Spatial zone classification of the robot workspace based on the sheet centroid position. *Source: own work.* 57
5.10 Three-level orientation strategy 62
5.11 Grip planning workflow 67
5.12 Opposite-face flip workflow 73

List of Tables

3.1 Functional Requirements for Automatic Robot Program Generation. . . 19
3.2 Non-Functional Requirements – NFR-01 to NFR-05 with acceptance criteria and verification methods. 20
3.3 Requirement Traceability Matrix — all requirements (FR-00 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios. 26

4.1 Structural comparison of Concept A, Concept B, and Concept C across key design features. 35
4.2 Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C. 39

5.1 Software modules across system layers and their primary responsibilities. 47
5.2 Failure Mode and Effects Analysis (FMEA) of the robotic bending system. 77
5.3 Requirement compliance matrix linking system requirements to implementation components. 81

6.1 Comparison between manual programming and automated system under varying conditions. 87

List of Code Snippets

5.1 Illustrative Python implementation of SHA-512-based authentication token computation following the drag&bot AnyRobot Interface specification [19]. 49
5.2 Generalised YAML configuration illustrating robot parameters, process definition, and motion and safety constraints. 51
5.3 Simplified Python implementation of grid-based grip candidate generation, geometric constraint filtering, and final grip position selection. . 68
5.4 Illustrative waypoint generation using minimum-jerk interpolation . . . 74
5.5 Illustrative determination of waypoint count based on path length . . . 74

List of Abbreviations

API	:	Application Programming Interface
CAD	:	Computer-Aided Design
CAM	:	Computer-Aided Manufacturing
Digital IO	:	Digital Input/Output
DNB	:	drag&bot
DOF	:	Degree of Freedom
FR	:	Functional Requirements
HI Dialog	:	Human Interaction (drag&bot-specific dialog service)
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IP	:	Internet Protocol
JSON	:	JavaScript Object Notation
LIN	:	Linear motion command
MAC	:	Message Authentication Code
NFR	:	Non-Functional Requirements
OLP	:	Offline Programming
PTP	:	Point-to-point motion command
ROS	:	Robot Operating System
SHA-512	:	Secure Hash Algorithm 512-bit
STL	:	Standard Tessellation Language
TCP	:	Tool Center Point
UI	:	User Interface
UR5e	:	Universal Robots 5e (robot model)
URDF	:	Unified Robot Description Format
WebSocket	:	Web-based communication protocol for real-time data exchange defined in RFC 6455
WSM	:	Weighted Sum Model
YAML	:	YAML Ain’t Markup Language

List of Symbols

A_{cup}	:	Contact area of one suction cup
a_{xy}	:	Resultant dynamic acceleration in the horizontal plane
a_z	:	Dynamic acceleration in the vertical direction
α	:	Euler angle corresponding to rotation about the Z-axis in the ZYX convention
β	:	Euler angle corresponding to rotation about the Y-axis in the ZYX convention
d_{cup}	:	Diameter of one suction cup
d_{face}	:	Reference-face angular offset used for orientation correction
D	:	Detectability rating in the FMEA
Δ_{ang}	:	Maximum angular deviation between current and reference fixture poses
Δ_{pos}	:	Euclidean positional deviation between current and reference fixture poses
ΔP	:	Pressure differential between ambient pressure and vacuum pressure
ΔRX	:	Rotational deviation about the X-axis
ΔRY	:	Rotational deviation about the Y-axis
ΔRZ	:	Rotational deviation about the Z-axis
Δx	:	Positional deviation in the X direction
Δy	:	Positional deviation in the Y direction
Δz	:	Positional deviation in the Z direction
dest	:	Destination field used during authentication
η	:	Efficiency factor accounting for leakage and surface conditions
F_{required}	:	Required holding force under gravitational and dynamic loading
F_{vacuum}	:	Available vacuum holding force
g	:	Gravitational acceleration
γ	:	Euler angle corresponding to rotation about the X-axis in the ZYX convention
id_string	:	Interface identifier used during authentication
λ	:	Interpolation parameter ranging from 0 to 1
level	:	Access level string used during authentication
m	:	Mass of the handled workpiece

MAC	:	Message authentication code generated for the AnyRobot authentication handshake
n	:	Number of active suction cups
nonce	:	Fixed nonce used during authentication
O	:	Occurrence rating in the FMEA
$\|$	:	Concatenation operator
$\mathbf{p}(\lambda)$	:	Interpolated translational position along a trajectory segment
$\mathbf{p}_s$	:	Translational component of the start pose
$\mathbf{p}_t$	:	Translational component of the target pose
R	:	Rotation matrix
R_{new}	:	Rotation matrix after applying the reference-face offset correction
R_{raw}	:	Raw rotation matrix from the tracked pose
R_x, R_y, R_z	:	Elementary rotation matrices about the X-, Y-, and Z-axes
R_{ij}	:	Element of a rotation matrix in row i and column j
RPN	:	Risk Priority Number in the FMEA
RX	:	Rotation about the X-axis (° or rad, depending on context)
RX'	:	Corrected or fallback rotation about the X-axis
RY	:	Rotation about the Y-axis (° or rad, depending on context)
RY'	:	Corrected or fallback rotation about the Y-axis
RZ	:	Rotation about the Z-axis (° or rad, depending on context)
RZ'	:	Corrected or fallback rotation about the Z-axis
RZ_{max}	:	Upper admissible limit for rotation about the Z-axis
RZ_{min}	:	Lower admissible limit for rotation about the Z-axis
S	:	Severity rating in the FMEA
SF	:	Safety factor for the required holding force
SHA-512	:	Hashing function used for generating the authentication token
secret	:	Shared runtime secret used during authentication
$s(u)$	:	Minimum-jerk progression function
$\mathbf{T}_s$	:	Start pose of a trajectory segment
$\mathbf{T}_t$	:	Target pose of a trajectory segment
t	:	Session timestamp used during authentication
u	:	Normalised progression variable for minimum-jerk interpolation
x	:	Translation along the X-axis (m)

x_{th}	:	Threshold value for workspace classification in the X direction (m)
y	:	Translation along the Y-axis (m)
y_{th}	:	Threshold value for workspace classification in the Y direction (m)
z	:	Translation along the Z-axis (m)

1 Introduction

1.1 Background and Motivation

Sheet metal bending on press brake machines is a common manufacturing process for producing functional sheet metal parts [1], [2]. In such processes, the bend sequence, tooling conditions, workpiece handling, and collision constraints are closely connected. When a robot is added to the press brake cell, the planning task becomes more demanding because grasp positions, repositions, and robot motion plans must also be considered [1], [2]. Although robotic systems can reduce direct manual handling, programming these systems still requires considerable engineering effort, especially when robot motions and process steps have to be adapted for new parts [3], [4].

In conventional robotic bending workflows, engineers often still define key poses, motion sequences, and process logic for each workpiece. This becomes inefficient when the sheet geometry, initial placement, or required bending sequence changes, because the robot program may need to be adjusted, checked, or recreated before execution [1], [2], [3].

The increasing need for flexible manufacturing highlights the importance of reducing manual intervention in robot programming. For robotic press brake bending, this means that program generation, grip selection, regripping decisions, and motion validation should be handled in a more systematic way. The motivation of this thesis is therefore to investigate how an automated planning pipeline can support faster and more repeatable preparation of robotic bending tasks [1], [3].

1.2 Problem Statement

Despite the use of robots in press brake bending, many workflows still depend on manually defined motion sequences and process logic. This creates high programming effort and makes the result strongly dependent on the experience of the programmer [3], [4].

A further limitation is the lack of automatic adaptation to changes in workpiece position, orientation, or geometry. In robotic bending, such changes influence grasp feasibility, reachability, collision conditions, and the required bending sequence. Without automated planning support, these changes often lead to manual adjustments and reduced flexibility [1], [2].

Regripping and multi-stage bending are especially difficult because they require coordinated decisions about where the workpiece is held, when it must be repositioned, and whether the next bend remains reachable. If these decisions are predefined manually, the system becomes harder to adapt to new workpieces or

process sequences [1], [2].
These limitations indicate the need for a systematic and automated approach that can generate robot programs dynamically while checking feasibility, safety, and consistency before execution.

1.3 Research Objectives

The primary objective of this thesis is to develop an automated system for robotic press brake bending that reduces manual programming effort while improving adaptability to varying workpiece conditions. The proposed approach aims to make the preparation of robotic bending tasks more structured by reducing operator intervention and by introducing systematic decision-making within the planning pipeline.
To achieve this objective, several specific goals are defined. First, the system is designed to automatically generate robot programs based on geometric input parameters, reducing the need for manual motion definition. In addition, the system enables adaptive handling of workpiece variations, including changes in position, orientation, and geometry.
A key aspect of the work is the integration of multiple planning components into a unified framework. These components include orientation determination, grip planning, regrip handling, and trajectory generation, all of which are essential for ensuring feasible and continuous execution of the bending process. Furthermore, the system incorporates structured validation mechanisms to ensure safe and collision-free operation during program execution.
Finally, the overall objective is to reduce operator dependency while maintaining consistent and repeatable system behaviour. By automating critical planning tasks, the proposed solution aims to support more flexible robotic bending workflows.

1.4 Scope of Work

This thesis focuses on the development and evaluation of an automated planning system for robotic press brake bending within a virtual simulation environment. The primary scope of the work is limited to the generation of robot programs and the validation of their execution feasibility using simulated data, without direct implementation on a physical robotic system.
The system is developed using a collaborative industrial robot from the Universal Robots family. This platform is selected because its payload, reach, repeatability, and available communication interfaces are suitable for developing and testing a structured robotic handling workflow in simulation [5].
The implementation is tailored to sheet metal bending tasks with defined geometric properties, allowing for controlled evaluation of the proposed planning approach. However, the work does not aim to address all possible industrial variations or com-

plex multi-process workflows, such as highly customised production scenarios or integration with additional manufacturing processes.
Furthermore, the evaluation is conducted entirely in a simulation environment, focusing on functional validation, feasibility assessment, and performance analysis. While this approach enables systematic testing and development, it does not include experimental validation on a physical robotic system, which is considered outside the scope of this thesis and a potential direction for future work.

1.5 Methodology Overview

The proposed solution follows a structured methodology that combines concept evaluation, modular system design, and systematic validation. The overall approach is designed to ensure that the developed system meets both functional and non-functional requirements while supporting reliable automated robot program generation.
Initially, multiple solution concepts are analysed and compared based on a set of defined evaluation criteria. These criteria include the ability to support automated program generation, as well as the extent to which each concept satisfies the identified functional requirements, non-functional requirements, and software design constraints. Based on this evaluation, the most suitable concept is selected for further development.
The selected concept is then implemented as a modular planning pipeline, consisting of distinct components responsible for orientation determination, grip planning, regrip handling, and trajectory generation. This modular structure enables clear separation of responsibilities and facilitates scalability, maintainability, and future extensions of the system.
To support robust system behaviour, the architecture is designed using a layered approach, ensuring that different levels of functionality are logically separated. Validation mechanisms are integrated at multiple stages within the pipeline to guarantee safe, consistent, and feasible execution of the generated robot programs.
Finally, the system is evaluated through a series of test scenarios to assess its functional correctness, performance, and adaptability to varying input conditions. A comparison with conventional manual programming approaches is conducted to quantify improvements in programming efficiency and execution time, thereby demonstrating the effectiveness of the proposed methodology.

1.6 Thesis Structure

The remainder of this thesis is structured as follows:
Chapter 2 reviews the state of the art in robotic press brake bending and identifies key limitations of existing approaches.
Chapter 3 defines the system requirements and outlines the methodology used for

concept development.
Chapter 4 presents the evaluation of alternative solution concepts and justifies the selection of the proposed approach.
Chapter 5 describes the implementation of the system, including architecture, planning modules, and validation mechanisms.
Chapter 6 evaluates the performance of the developed system through functional validation, system-level analysis, and comparison with manual programming approaches.
Finally, Chapter 7 summarises the contributions of the thesis and discusses limitations and directions for future work.

2 State of the Art and Literature Review

2.1 Fundamentals of Industrial Robotics

Industrial robots are a central part of modern manufacturing because they combine repeatability, precision, and the ability to perform physically demanding operations over long production cycles. The international standard ISO 8373 describes an industrial robot as a reprogrammable and multipurpose manipulator with three or more programmable axes [6]. In practice, such systems are used for handling, assembly, welding, forming, and many other operations in which stable and repeatable motion is required.

A typical industrial robot consists of several actuated joints that provide the degrees of freedom (DOF) needed to position and orient an end-effector in three-dimensional space. The task-relevant reference point is usually defined by the Tool Center Point (TCP), which links the robot motion to the physical tool or gripper. Forward and inverse kinematics describe the relationship between joint variables and the Cartesian pose of this reference point, forming the basis for motion planning and robot control [7].

Robot programming can be understood as a spectrum of abstraction levels. At the lower end, the operator or programmer explicitly defines poses, trajectories, and process sequences. At higher levels, the user specifies the intended task, while the system derives executable actions and motion commands. Industrial practice still relies heavily on manual and offline programming, but research has increasingly moved toward higher levels of abstraction, including task-level programming and skill-based approaches that reduce the need for detailed low-level motion specification [3], [8].

This shift is especially important for manufacturing tasks that are not simple repetitions of a fixed motion. In applications such as robotic sheet metal bending, the robot must handle changing workpiece geometry, limited workspace, tool access constraints, and possible reorientation steps. These characteristics make the task more demanding than conventional pick-and-place operations and motivate the need for a broader review of robot programming methods, press brake automation, and integrated planning approaches.

2.2 Sheet Metal Bending and Press Brake Systems

Sheet metal bending is a forming process in which a flat metal sheet is plastically deformed along one or more bend lines. In a press brake, the sheet is positioned between a punch and a die, and the bending result depends on tool geometry, material properties, sheet thickness, bend sequence, and applied force [9]. The

process is common in industrial production because it can create a wide range of functional parts from relatively simple sheet blanks.
Figure 2.1 illustrates a typical robotic press brake setup, where an industrial robot is integrated with the bending machine to automate handling operations.



Figure 2.1: Robotic press brake bending system with integrated robot handling (Source: [10]).

In automated press brake cells, the robot usually picks the flat sheet, positions it at the tooling, supports it during bending, reorients it between process steps, and unloads the final component. This automation can improve repeatability and reduce direct operator involvement, particularly in repetitive production settings [2]. However, the process is difficult to automate fully because each bend changes the physical shape of the part.

2.2.1 Process Characteristics and Automation Challenges

The changing geometry of the workpiece is one of the defining characteristics of sheet metal bending. A motion that is feasible before one bend may become infeasible after the sheet has been formed, because the part envelope, tool accessibility, and collision conditions have changed. Process planning must therefore consider the bend sequence, the accessibility of the tooling, and the handling strategy together rather than as isolated decisions [1].
For robotic handling, these process characteristics lead to several practical challenges. The robot must maintain a stable grasp, avoid collisions with the press brake and the partially formed workpiece, remain inside its reachable workspace, and preserve a suitable orientation for the next operation. In many cases, a single grasp is not sufficient for the full bending sequence, so the workpiece must be placed down and grasped again in a different configuration. These issues are introduced here

as process-level constraints; the detailed planning problems of regripping, grasp selection, and trajectory generation are discussed later in Section 2.6.

2.3 Manual and Teach Pendant Programming

Manual programming with teach pendants is one of the most established methods for programming industrial robots. In this approach, an operator defines the robot motion by jogging the robot to desired positions or by entering Cartesian or joint coordinates. The resulting positions are stored as waypoints and executed in a defined sequence by the robot controller [11].
Figure 2.2 illustrates a typical teach pendant interface used for manual robot programming.

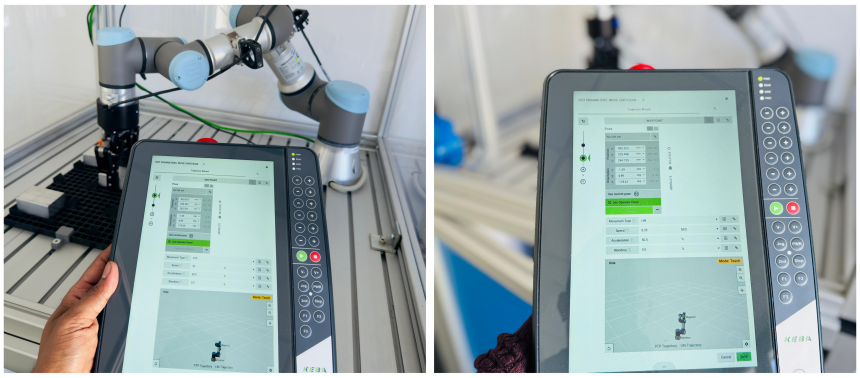


Figure 2.2: Teach pendant interface for manual robot programming (Source: own work).

Teach pendant programming remains useful because it gives the operator direct control over the robot and allows immediate verification of individual motions. For stable, repetitive, and well-known tasks, this can be a practical and reliable method. Its limitations become more visible when the process requires many coordinated motions or frequent adaptation to new workpieces.

2.3.1 Limitations of Manual Programming

The main limitation of manual programming is the amount of expert effort required. The operator must understand the robot coordinate system, tool frame, reachability limits, process constraints, and possible collision situations. As the number of waypoints grows, the programming task becomes slower and more dependent on individual experience.
Manual programming is also poorly suited to frequent product changes. If the geometry of the workpiece changes, the robot program often has to be adapted or recreated, even when the process logic remains similar. This makes manual programming inefficient for flexible manufacturing environments with high part variation [3]. In addition, reachability, collision avoidance, and path quality are not

automatically guaranteed; they must be checked and corrected by the programmer.

2.3.2 Challenges in Complex Manipulation Tasks

Robotic sheet metal bending exposes these limitations clearly. The task is not a single transfer motion, but a sequence of handling, positioning, bending, reorientation, and possibly regripping steps. The operator must anticipate how the sheet will change after each bend and must define motions that remain feasible throughout the process.
Regripping is particularly difficult to program manually because it requires coordination between placement, release, new grasp selection, and the next bending operation. Small changes in part dimensions or bend order can alter the required intermediate positions. As a result, manual programming can become time-consuming, difficult to reuse, and vulnerable to inconsistent decisions between operators.

2.3.3 Transition Towards Automated Programming

The limitations of manual programming have encouraged the development of methods that raise the level of abstraction. Instead of manually defining every pose, engineers increasingly use virtual commissioning, offline programming, task-level programming, and rule-based planning to reduce direct interaction with the robot. These approaches do not remove all engineering effort, but they shift part of the work from manual waypoint teaching toward model-based planning and validation.

2.4 Offline Programming Approaches for Industrial Robots

Offline programming (OLP) enables robot programs to be prepared and tested in a virtual environment before they are executed on the physical robot. In contrast to teach pendant programming, OLP decouples programming from production, allowing engineers to develop, simulate, and validate robot tasks without occupying the real workcell [3].
Figure 2.3 illustrates an offline programming environment, where a virtual model of the robot, tooling, and workcell is used to simulate and validate motion trajectories. OLP systems commonly use geometric models of the robot, tooling, fixtures, and surrounding equipment. These models allow engineers to evaluate reachability, motion paths, and possible collisions before deployment. In this way, offline programming can reduce floor trial-and-error and support earlier detection of workcell or path problems before production is interrupted [3], [12].

2.4.1 Digital Twin and Simulation-Based Programming

Digital twin concepts extend simulation-based programming by linking the virtual representation more closely to the physical system. A digital twin can support mon-

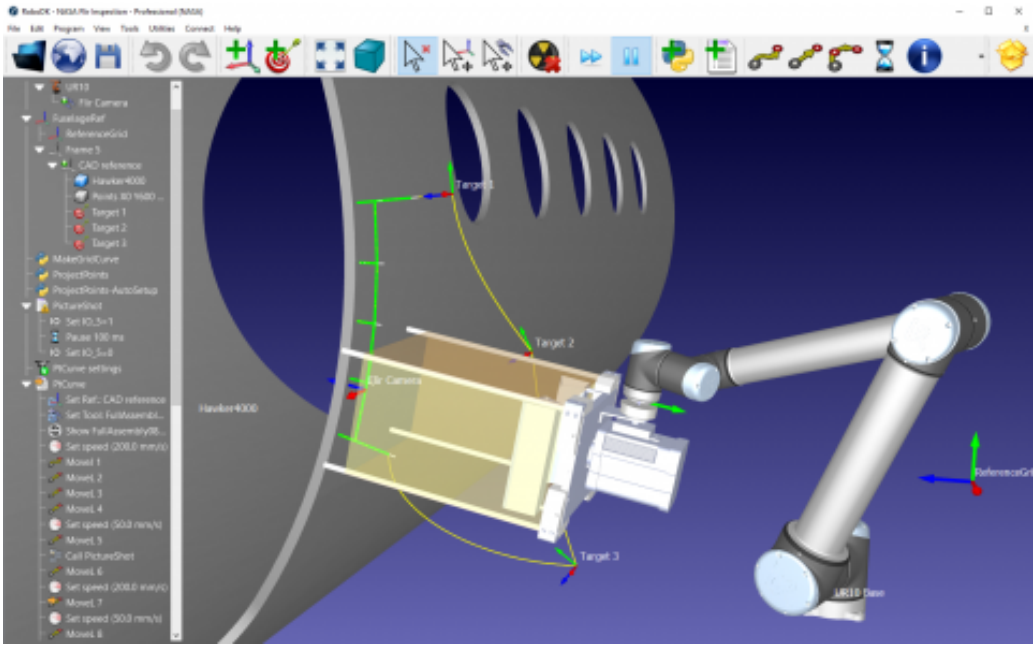


Figure 2.3: Offline robot programming environment with CAD-based simulation (Source: [12]).

itoring, validation, and adjustment by combining model-based simulation with data from the real process. In manufacturing, digital twin approaches have been discussed as an important element of smart production systems because they support analysis, synchronisation, and cyber-physical integration [13].

In industrial robotics, tools such as RoboDK, ABB RobotStudio, KUKA.Sim, and drag&bot provide virtual environments for programming and validation. They support tasks such as visualising the workcell, checking reachability, identifying collisions, and generating executable robot code. These capabilities make simulation-based programming valuable for complex cells where direct trial-and-error programming would be slow or risky.

2.4.2 Limitations of Offline Programming

Despite these advantages, offline programming does not automatically solve the programming problem. The quality of the generated program depends strongly on the accuracy of the virtual model. Differences between the simulated workcell and the real cell can lead to deviations during execution, requiring calibration or manual correction.

Another limitation is that many OLP tools still require the engineer to define the task sequence, waypoints, and process logic manually. The software can validate and simulate a program, but it does not necessarily decide which grasp to use, when to regrip, or how to generate a complete strategy for a changing workpiece. This limits the usefulness of conventional OLP for processes such as robotic sheet metal bending, where the task geometry and constraints evolve during execution.

2.5 Automatic Robot Program Generation and Rule-Based Approaches

Automatic robot program generation aims to reduce the need for manual waypoint definition by deriving robot actions from task descriptions, process parameters, geometric input, or formal rules. Instead of programming every motion explicitly, the system computes feasible actions based on the available information and the constraints of the process.

2.5.1 Task-Level Programming and Abstraction

Task-level programming is based on the idea that users should describe what the robot should achieve, while the system determines how the task can be executed. Early work on robot programming introduced abstraction layers and symbolic task descriptions as a way to separate task intent from low-level robot motion [14].
More recent approaches use skills, motion primitives, and reusable task modules. A skill may represent an operation such as approaching, grasping, placing, or moving along a constrained path. By combining such skills, complex robot programs can be assembled in a more modular way [8]. This improves reusability, but the system still needs reliable mechanisms for selecting valid actions and checking whether the resulting motion is feasible.

2.5.2 Rule-Based and Constraint-Based Planning

In structured industrial environments, automatic program generation is often implemented through rules and constraints. Rules may describe how a workpiece should be approached, which orientations are allowed, when a regrip is necessary, or which motion types should be used. Constraint-based planning extends this idea by explicitly evaluating geometric, kinematic, and process-related conditions before selecting a solution.
Such methods are attractive because they can produce deterministic behaviour in well-defined applications. However, when product variants and process constraints increase, the engineering effort required to define, adapt, and validate these rules also increases. In robotic sheet metal bending, this issue is especially relevant because bend sequencing, tooling, grasp selection, and robot motion planning are closely coupled [1], [3].
For complex manipulation tasks, the main challenge is not only to generate isolated decisions, but to coordinate several planning components. Grasp planning, regrip planning, trajectory generation, reachability checking, and collision avoidance influence each other. If these components are developed separately, a locally valid decision may still lead to an infeasible overall process. Robotic manipulation research therefore emphasizes the importance of integrated planning frameworks, especially when grasping and motion constraints are tightly coupled [15].

2.6 Robotic Sheet Metal Bending Systems

Robotic sheet metal bending systems combine an industrial manipulator with a press brake to automate the handling and positioning of sheet metal parts. The robot transfers the sheet, aligns it with the tooling, supports it during bending, and prepares it for subsequent process steps. This integration can improve repeatability and reduce direct manual handling, but it also creates a planning problem that is more complex than ordinary material transfer.
Figure 2.4 illustrates a typical robotic bending setup, where the robot operates in coordination with the press brake to execute a sequence of manipulation and forming operations.

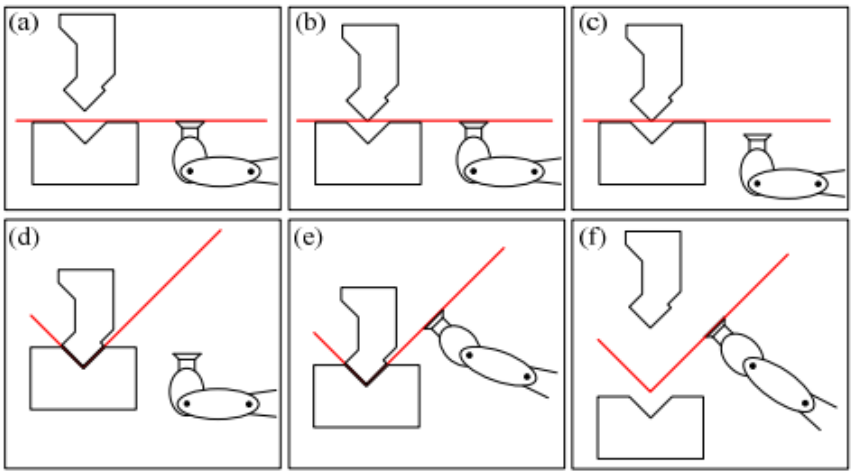


Figure 2.4: Robotic sheet metal bending system with integrated robot and press brake (Source: [4]).

The process usually starts with a flat sheet, but after every bend the shape of the workpiece changes. This affects collision boundaries, grasp accessibility, tool clearance, and reachable robot configurations. Research on robotic bending has therefore shown that successful automation depends on coordinating process planning with robot motion planning [1]. The following subsections discuss the main planning aspects that are relevant for this thesis.

2.6.1 Regrip Planning in Sheet Metal Bending

Regripping is often necessary when the robot cannot execute all bending steps with a single grasp. The required orientation for later bends may conflict with the initial grasp, or the formed geometry may make the original grasp unsuitable. In such cases, the robot places the workpiece at an intermediate location, releases it, and grasps it again in a new configuration.
From a robotics perspective, regrasp planning can be formulated as a search over feasible grasps and transitions between them. A valid sequence must consider grasp stability, reachability, collision avoidance, and the availability of intermediate placements [16]. In industrial bending cells, however, regripping is often handled

through predefined sequences or manually designed rules. This can be reliable for known parts, but it reduces flexibility when the workpiece geometry or bend order changes.

2.6.2 Challenges of Regrip Integration in Industrial Systems

The difficulty of regripping is not limited to selecting a new grasp. A regrip decision affects the following trajectory, the accessibility of the next bend, and the risk of collision with both the workpiece and the press brake. Therefore, regrip planning cannot be treated independently from motion planning. In robotic press brake process planning, tooling, grasping, and moving are coupled planning tasks, so a valid bending strategy has to be checked across the process sequence rather than at one isolated step [1].
In many existing workflows, grasp selection, regrip decisions, and trajectory validation are performed as separate steps. This separation can create practical problems: a grasp may be stable but unreachable, a regrip pose may be geometrically valid but difficult to approach, or a trajectory may become infeasible after the part has been bent. These dependencies show why integrated planning is important for scalable robotic bending.

2.6.3 Grasp Planning for Sheet Metal Handling

Sheet metal parts are challenging to handle because they are thin, often large relative to their thickness, and may deform or slip if the grasp is poorly chosen. Vacuum grippers are widely used because they can hold flat sheet surfaces without requiring mechanical clamping around the part. Their performance depends on suction cup placement, contact quality, available holding force, and the distribution of loads during motion [17].
General grasp planning methods consider object geometry, contact conditions, and task constraints when selecting stable grasp configurations [15]. In sheet metal bending, the selected grasp must also leave the bend line and tool access region free. A grasp that is suitable for picking the flat sheet may interfere with a later bending operation, especially after the workpiece geometry has changed. As a result, grasp planning must be linked to the process sequence and not only to the initial sheet geometry.

2.6.4 Trajectory Planning in Robotic Bending Systems

Trajectory planning in robotic bending must produce motions that are collision-free, reachable, and compatible with the process constraints. The robot operates close to the punch, die, workpiece, and other cell components, leaving limited room for large avoidance motions. At the same time, it must maintain a suitable orientation so that the sheet can be positioned accurately at the tooling.

Classical trajectory generation methods provide useful foundations for creating smooth and continuous robot motions [18]. However, robotic bending introduces additional constraints because the workpiece geometry changes during the process. The planner must account for evolving collision conditions, workspace limits, possible singularities, and the need to coordinate motion with grasp and regrip decisions [1]. In many industrial systems, these challenges are handled by conservative motion templates or manually defined waypoints, which improves reliability but limits adaptability.

2.6.5 Limitations of Existing Robotic Bending Systems

Existing robotic bending systems have demonstrated that press brake automation is technically feasible, but several limitations remain. Many systems rely on predefined handling strategies and carefully engineered programs. This makes them effective for stable production scenarios, but less suitable for flexible environments where part geometry and process requirements change frequently.

A central limitation is the separation of planning tasks. Grasp planning, regrip planning, trajectory generation, and process validation are often treated as separate activities. This can produce fragmented workflows in which a decision that is valid in one module creates problems in another. For example, a selected grasp may be stable but unsuitable for the next trajectory, or a manually defined path may work for one part variant but fail when the geometry changes.

Another limitation is the continued dependence on expert intervention. Even when simulation and offline programming tools are available, engineers commonly define motion sequences, check reachability, adjust waypoints, and validate process logic manually. This increases setup time and makes adaptation to new parts more difficult. These limitations point toward the need for a unified approach that can combine constraint-aware planning, automated trajectory generation, and structured execution.

2.7 Industrial Software Platforms and System Implementations

Industrial robot applications are commonly developed with software platforms that support programming, simulation, validation, and execution. These platforms bridge the gap between theoretical planning methods and practical deployment by giving engineers structured tools for building and testing robotic workflows.

2.7.1 Simulation and Offline Programming Environments

Simulation and offline programming environments allow robot programs to be prepared, simulated, and generated before they are executed in the physical cell. They can support reachability checks, collision evaluation, motion visualisation, and the generation of controller-specific robot code, although the exact capability depends

on the software platform [3], [12]. These capabilities reduce commissioning risk and make it easier to identify errors before production is interrupted.
Nevertheless, such environments are usually not fully automatic planning systems. They provide powerful tools for validation and development, but the user often remains responsible for defining the task sequence, selecting key poses, and configuring the logic that connects individual operations. For complex manipulation processes, this manual configuration remains a significant part of the engineering effort.

2.7.2 Graphical Programming and Integrated Execution Systems

Graphical programming platforms such as drag&bot further reduce the barrier to robot programming by allowing users to assemble workflows from modular function blocks [19]. Such systems can combine user interaction, robot communication, and runtime execution in one environment. They are useful for implementing and operating robotic applications without requiring extensive low-level programming.
At the same time, graphical execution systems usually depend on workflows and logic that are defined by the user. They can execute and monitor a process, but they do not inherently solve the planning problem of deciding where to grasp, when to regrip, or how to generate a complete trajectory sequence for a changing workpiece. This distinction is important for robotic bending, where the main difficulty lies in coordinating several dependent planning decisions.

2.7.3 Relevance to This Thesis

Existing industrial platforms provide a useful foundation for simulation, communication, and execution. Their limitation is not that they lack practical value, but that they still leave much of the task strategy to the engineer. For press brake bending, this means that grasp selection, regrip handling, and trajectory generation often remain manually configured or only partially automated.
The approach developed in this thesis builds on the strengths of such platforms while adding an automated planning layer. The aim is not to replace industrial execution environments, but to reduce the amount of manual task definition required for robotic bending. By generating and validating motion-relevant decisions within a structured pipeline, the system can use existing runtime tools while improving adaptability to changing workpiece conditions.

2.8 Research Gap and Motivation

The literature shows a clear development from manual robot programming toward simulation-supported and partially automated methods. Manual programming remains reliable for stable tasks, but it does not scale well when workpiece geometry changes frequently. Offline programming improves safety and commissioning ef-

ficiency, yet it still depends on user-defined task sequences and accurate virtual models. Automatic programming and rule-based methods reduce manual effort further, but many approaches focus on individual decisions rather than on the complete bending workflow.

In robotic sheet metal bending, this separation is a major obstacle. The workpiece changes shape after each bend, and this affects grasp feasibility, regrip requirements, collision conditions, and reachable trajectories. A planning decision cannot be evaluated properly if it is isolated from the rest of the process. A grasp that is acceptable at one stage may block tool access later; a regrip pose may be geometrically possible but difficult to reach; a trajectory may be smooth in isolation but unsuitable once the formed part is attached to the robot.

The resulting research gap is the lack of a practically deployable workflow that integrates grasp planning, regrip planning, and constraint-aware trajectory generation for robotic press brake bending. Existing methods and tools provide important foundations, but they do not sufficiently address these interdependent planning tasks as one coordinated process under workspace, kinematic, and process constraints.

Addressing this gap is important for flexible automation. A system that can generate valid target poses, evaluate process constraints, and transform these decisions into executable robot motions can reduce engineering effort and improve adaptability. This thesis therefore proposes an integrated and constraint-aware approach in which automated trajectory generation, workspace-aware planning, and deterministic execution are combined within a structured pipeline.

The identified gap and the proposed direction form the basis for the concept development presented in the following chapter.

3 System Concept and Methodology

3.1 Problem Definition and Objectives

Industrial robots are already used in press brake sheet metal bending, but in many cases they are still programmed manually or through predefined routines. Those approaches work well as long as part geometry, pickup conditions, and bend sequences remain stable. Their limitations become much more visible in flexible production environments, where frequent part variation leads to repeated manual adjustment, longer setup times, and reduced efficiency.

The main technical challenge comes from the changing geometry of the workpiece. After each bending operation, the shape of the part changes, and that change affects robot reachability, grip feasibility, valid grip locations, and the set of trajectories that can still be executed safely. Handling and regripping therefore cannot be treated as isolated steps. They have to be planned in relation to the full process sequence, which makes robotic bending difficult to scale when programming is performed largely by hand.

From a methodological perspective, conventional robot programming still operates at a relatively low level of abstraction. Paths, tool settings, and waypoints are defined explicitly, which ties the resulting program closely to one specific robot setup, one grip configuration, and one process sequence. Programs created in this way are difficult to adapt when reorientation or regripping becomes necessary, and they scale poorly in manufacturing settings that require frequent adjustment [3].

Research has already explored higher-level programming approaches that bring planning and motion execution closer together. Even so, these ideas have not been transferred fully into press brake bending scenarios in which changing workpiece geometry and handling conditions must be considered explicitly. A clear gap therefore remains between general robot-programming research and a practical workflow for robotic sheet metal bending.

Based on these observations, the central problem addressed in this thesis can be stated as follows:

There is no systematic method for automatically generating robot programs for press brake sheet metal bending that explicitly considers changing workpiece geometry, handling feasibility, and regripping requirements while also reducing manual programming effort.

The main objective of this thesis is therefore to enable the automatic generation of robot programs for sheet metal handling and bending operations in a way that reduces manual effort and improves adaptability to varying workpiece geometries.

To address this objective, this work develops a conceptual and methodological framework that brings bending, handling, regripping, and orientation management

together in one workflow. The intention is to generate robot trajectories and execution logic automatically from process inputs, without relying on manually defined waypoint sequences.
The specific objectives of this work are:

- Develop a method for the automatic generation of robot programs based on workpiece geometry and process parameters.
- Generate feasible robot trajectories while considering geometric, kinematic, and workspace constraints.
- Support multi-stage bending operations, including reorientation and regripping of the workpiece.
- Develop a method for determining feasible grip points based on geometry, accessibility, and process stage.
- Enable integration with an industrial robot programming environment through a communication interface.
- Reduce manual programming effort while ensuring consistent and reliable system behaviour.

The proposed approach is evaluated in a virtual simulation environment compatible with industrial robot systems. Accordingly, the scope of this thesis is limited to conceptual development and simulation-based validation.

3.2 Requirement Definition and Management

The system concept developed in this thesis is guided by a defined set of requirements derived directly from the problem outlined in Section 3.1. These requirements specify the expected functionality, describe the intended system behaviour, and make the operational boundaries explicit.
To keep the development process transparent, the requirements are organised into functional, non-functional, system-level, and software-specific categories. This structure supports consistent design, implementation, and validation and also makes later traceability easier to establish.

3.2.1 Functional Requirements

The functional requirements define the capabilities that the system must provide in order to generate robot programs automatically for sheet metal bending operations. In practical terms, they describe how the system processes input data, produces robot motion, and maintains feasibility over several stages of the bending process. The most important requirement is the automatic generation of robot programs from operator-defined inputs. This directly addresses the limitations of manual

programming identified in the previous chapters and forms the core of the proposed solution.
Meeting that requirement involves more than creating a nominal path. The system has to generate feasible robot trajectories for handling and bending operations on the basis of target poses and process parameters, and those trajectories must satisfy the relevant geometric and kinematic constraints within the defined workspace. Because the shape of the sheet changes after each bending step, the system must also support multi-stage processes that include reorientation and regripping. Grip points cannot be fixed once at the beginning of the process; they have to be determined according to the current process stage, accessibility, and workpiece state so that stable handling remains possible throughout the sequence.
All generated motions must also be validated before execution. If infeasible conditions are detected, such as unreachable poses or invalid configurations, the system must either trigger replanning or terminate execution safely when recovery is not possible.
Finally, the complete program must be checked in simulation before execution. This makes it possible to verify the generated sequence in a controlled environment and to confirm that the solution remains consistent with the defined system constraints.
The detailed functional requirements corresponding to these capabilities are summarised in Table 3.1.

3.2.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes that the system must exhibit in addition to its basic functionality. They define how the system should behave under use and therefore cover aspects such as usability, maintainability, robustness, and repeatability.
One important design goal is that operators should be able to define process inputs without detailed knowledge of low-level robot programming. In that sense, usability is not treated as an afterthought, but as a condition for practical adoption.
The system is also expected to behave consistently. For identical input conditions, it should produce deterministic results. In an industrial setting, this kind of predictability is essential because repeatable behaviour supports both safe execution and reliable process evaluation.
Maintainability is addressed through a modular system structure. Planning, validation, and execution are treated as distinct responsibilities so that individual components can be developed, maintained, and extended without forcing unnecessary changes across the whole system.
Robustness is equally important. Within the defined scope, the system should tolerate minor variation in inputs and process conditions while still producing stable output. In addition, the approach should remain transferable to comparable process configurations and workpiece variations inside the intended application range.
The detailed non-functional requirements corresponding to these quality attributes

ID	Functional Requirement (SMART)	Acceptance Criteria	Verification Method
FR-00	Automatic robot program generation shall be implemented for press brake operations.	The system shall generate robot programs automatically for press brake operations.	Simulation based
FR-01	The system shall detect unreachable target poses before robot motion execution.	The robot shall not execute motion to unreachable poses and shall provide an error message.	Simulation based
FR-02	The system shall allow operator-side selection prior to program execution.	The selected bend side shall be applied correctly to the generated robot program.	Simulation based
FR-03	The system shall execute a regripping sequence when a target pose is unreachable within the current bend side.	Regripping shall be performed successfully and execution shall continue without interruption.	Simulation based
FR-04	The system shall handle workpiece orientation changes during program execution.	Orientation changes shall be applied correctly without violating kinematic limits.	Simulation based
FR-05	The system shall support opposite-side bending operations when required.	The system shall correctly execute opposite-side bending operations.	Simulation based
FR-06	The system shall determine and apply suitable grip positions for each workpiece state.	Grip positions shall be determined and applied correctly for each workpiece state.	Simulation based
FR-07	Detect unsafe conditions and perform replanning or terminate execution	System replans successfully with warning	Simulation based

Table 3.1: Functional Requirements for Automatic Robot Program Generation.

are summarised in Table 3.2.

Table 3.2: Non-Functional Requirements – NFR-01 to NFR-05 with acceptance criteria and verification methods.

ID	Non-Functional Requirement	Acceptance Criteria	Verification Method
NFR-01	The system shall minimise operator interaction during normal execution. Configuration steps prior to execution shall be limited to essential parameters.	Operator interaction limited to bend-side selection and execution confirmation.	Usability validation
NFR-02	The system shall allow process parameters, handling strategies, and system settings to be modified through configuration mechanisms that are fully separated from the core program logic, with no source code changes required. Sufficient terminal logging shall be provided to support systematic debugging, monitoring, and future extension of the system.	New workpiece geometry configurable via YAML file update only, without source code modification. Configuration change validated without rebuild.	Configuration validation test
NFR-03	The system shall provide clear warning and error messages for unsafe or infeasible conditions without requiring expert robot programming knowledge.	Warning and error messages displayed for all detected unsafe conditions, with controlled continuation or termination.	Warning message validation
NFR-04	The system shall exhibit robust behaviour under repeated execution cycles. Small variations in workpiece orientation or timing shall not lead to system failure or unsafe behaviour. The system shall handle deviations in a controlled manner.	System completes at least five consecutive bending cycles without failure under nominal variation conditions.	Repeated run validation test
NFR-05	The system shall tolerate moderate pose deviations within defined limits without triggering an error or aborting execution. Deviations exceeding the threshold shall trigger a warning and controlled stop.	Pose deviations within the defined tolerance threshold do not cause execution termination. Deviations exceeding the threshold trigger a warning and controlled stop.	Perturbation validation test

3.2.3 System Constraints and Assumptions

The proposed system is developed within a deliberately limited set of constraints and assumptions. These boundaries define the scope of the work and help keep the resulting behaviour predictable and reproducible.

At the geometric level, the workpiece model is restricted to rectangular sheet metal components with exactly four edges aligned to the global coordinate system. Rotated or non-axis-aligned sheet configurations are excluded. A single global wall height parameter is assumed for all bending operations, so edge-specific wall-height variation is not supported. Internal features are represented only as circular holes defined by centre coordinates and radius; more complex shapes such as polygonal cut-outs or irregular contours are outside the present scope.

The bending process is simplified in a similarly explicit way. Only right-angle ($90°$) bends are considered, and user-defined bend angles are not supported. Each sheet edge can be assigned to only one bending operation, which creates a one-to-one relationship between edges and bends and keeps the sequence deterministic. The number of bends is also limited to four on each side of the workpiece, including both primary and opposite-side bending stages.

On the system side, the robot configuration, tooling setup, and workspace layout are assumed to remain fixed during operation. Motion generation relies on a predefined tool centre point configuration, and all generated trajectories are expressed relative to that reference. The work also assumes a vacuum-based gripping mechanism with a predefined input-output configuration. Changes in tool geometry, TCP definition, or wiring are not handled dynamically and therefore require manual reconfiguration.

The execution environment is likewise assumed to remain structured and predefined. The workflow depends on consistent identification and positioning of workpieces and support structures, so deviations from the expected setup may lead to incorrect execution behaviour. For that reason, the present implementation remains tightly coupled to the defined virtual cell.

With respect to gripping, only vacuum-based strategies are considered. Grip feasibility is evaluated using simplified physical considerations such as pressure distribution, friction, and load capacity. Alternative gripping methods, including mechanical, magnetic, or multi-fingered grippers, are excluded.

Finally, the scope of the work is limited to rule-based program generation, geometric reasoning, and simulation-based validation. More advanced capabilities, such as dynamic collision-free trajectory generation, real-time motion adaptation, and environment-aware path optimisation, are intentionally left out. This keeps the system aligned with the thesis focus on deterministic automatic program generation in a controlled industrial setting.

3.2.4 Software Platform and Tool Requirements

The software platform and tool requirements describe the technical environment in which the proposed system is implemented and evaluated. They define the execution context, the communication structure, and the reference conditions needed to support automated program generation, validation, and simulated execution.
These requirements are grouped as SR-01 to SR-03 and are detailed in the following subsections.
At a general level, the system has to operate within a virtual robot programming environment that supports simulation, execution, and validation of robot programs. In this thesis, drag&bot is used as that environment because it provides a structured setting for testing generated trajectories under controlled conditions.
The planning module must also communicate with the execution environment through one unified interface. This role is assigned to the AnyRobot interface, which acts as an abstraction layer between the planning logic and the robot programming environment. Communication is based on the Web-based communication protocol for real-time data exchange defined in RFC 6455 (WebSocket) protocol and supports bidirectional exchange of target poses, execution commands, and feedback data.
All generated robot motion targets must further be defined with respect to a predefined TCP configuration. This is necessary to maintain consistency between the planning calculations and the robot representation in the execution environment.
Beyond that, the software architecture must support structured internal data exchange for process inputs, geometric representations, generated trajectories, and validation results. The implementation therefore follows a modular architecture that separates planning, validation, and execution concerns.
Simulation-based validation prior to execution is also required at the software level. In this way, the generated trajectories can be checked against process constraints before runtime execution is attempted.

3.2.4.1 SR-01 Virtual Execution Environment

The system shall operate within a virtual robot programming environment that supports simulation, execution, and validation of robot programs. In this work, the drag&bot platform is used as the designated runtime environment for executing and validating all generated robot trajectories.
The drag&bot environment provides a unified interface for robot motion execution, object handling, and input-output operations, allowing the planning system to interact with the robot model in a controlled and structured manner. This ensures that all generated programs can be evaluated consistently within a virtual representation of the target press brake cell without requiring physical hardware access.
By operating within this environment, the system enables reliable simulation-based validation of robot programs, ensuring that functional requirements related to tra-

jectory generation, handling, and process sequencing are verified prior to execution. The use of a virtual runtime environment also supports reproducibility of results and reduces dependency on physical system availability.

Acceptance Criteria - SR-01 The system shall establish a stable connection to the virtual runtime environment, and all generated robot motions, service calls, and input-output operations shall be executed and validated successfully within the simulation environment.

3.2.4.2 SR-02 Communication Interface Requirement

The system shall communicate with the robot execution environment through a unified communication interface that enables structured interaction between the planning module and the runtime system. In this work, the AnyRobot interface is used as the exclusive communication layer between the planning system and the execution environment.
The communication shall be implemented using a WebSocket-based protocol, enabling bidirectional data exchange for transmitting robot motion commands, input-output signals, object handling operations, and system feedback. This approach provides a consistent abstraction layer for interacting with the robot programming environment without requiring direct access to robot-specific control interfaces.
By using a unified communication interface, the system ensures that all commands and execution requests are transmitted in a structured and controlled manner, supporting maintainability and compatibility across different system configurations. This abstraction also enables the planning system to remain independent of specific robot controller implementations.

Acceptance Criteria - SR-02 All communication between the planning module and the execution environment shall occur through the defined communication interface. A valid connection shall be established before command transmission, and all motion, input-output, and service operations shall be successfully executed and acknowledged within the simulation environment.

3.2.4.3 Tool Centre Point Definition Requirement SR-03

The system shall define all robot motion targets with respect to a predefined TCP configuration. The TCP represents the reference frame of the end-effector and is used as the basis for all trajectory generation and motion planning operations.
All generated robot poses, including approach positions, handling configurations, and placement targets, shall be expressed in relation to the TCP. This ensures consistency between the planning module and the execution environment and guarantees that the generated trajectories correspond to the physical geometry of the tool.

The TCP configuration is defined externally and remains fixed during system operation. Any changes to the tool configuration or TCP definition require corresponding updates to ensure alignment between the generated trajectories and the execution environment. This requirement ensures that motion accuracy and handling feasibility are maintained throughout the process.

Acceptance Criteria - SR-03 All generated robot motion targets shall be consistent with the predefined TCP configuration, and any modification to the TCP definition shall be reflected correctly in the generated trajectories without requiring structural changes to the system logic.

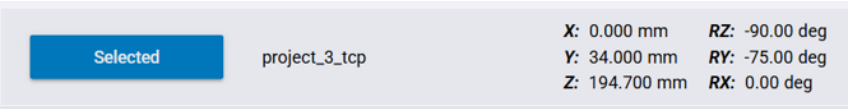


Figure 3.1: Tool Center Point parameters in drag&bot.

3.2.5 System Traceability

Requirements traceability is essential in a systematic engineering process because it makes the relationship between requirements, implementation, and validation explicit. In the present work, traceability is used to show how the transition from conceptual objectives to executable system behaviour can be followed and verified. Each requirement defined in Section 3.2.1, Section 3.2.2, and the software requirements SR-01 to SR-03 is assigned a unique identifier such as FR-01, NFR-01, or SR-01. These identifiers are then used consistently throughout development so that requirements can be tracked across design, implementation, and validation. The identifiers establish clear relationships between requirements, system modules, and validation procedures. In particular:

- Each requirement is implemented within a specific system module.
- Each requirement is verified through a defined test scenario.
- No requirement remains unaddressed in the implementation.
- All implemented functionalities can be traced back to a corresponding requirement.

Forward Traceability Forward traceability means that every defined requirement is realised in a corresponding system component or functional module. It confirms that the system architecture addresses all stated requirements.

Backward Traceability Backward traceability means that every implemented functionality can be linked back to an explicitly defined requirement. This prevents unnecessary or unjustified functionality from being introduced without a clear purpose.

By mapping each requirement to implementation modules and validation scenarios, the development process becomes easier to review, justify, and reproduce. This is particularly important in a system that includes automated decision-making steps such as reachability evaluation, grip-point determination, regripping logic, and opposite-side handling.

Traceability also supports maintainability and future extension. If requirements change, the affected components and validation activities can be identified directly, which makes impact analysis more efficient and reduces development risk.

The complete mapping between requirements, system modules, and validation scenarios is presented in Table 3.3.

Req. ID	Requirement Description	Implemented In	Verified In
FR-00	Automatic robot program generation for press brake operations.	Integrated planning pipeline combining geometry processing, trajectory generation, and execution logic	End-to-end simulation validation
FR-01	Detect unreachable target poses before execution.	Reachability checking and admissibility validation logic	Simulation test case TC-01
FR-02	Allow operator-side selection of bending configuration before execution.	Operator dialog and configuration collection logic	Execution test case TC-02
FR-03	Execute regripping sequence when a target pose is unreachable.	Automatic regrip cycle logic with I/O and attach/detach control	Regrip validation test case TC-03
FR-04	Handle workpiece orientation changes during program execution.	Orientation conversion, rotation normalisation, and strategy selection logic	Orientation validation test case TC-04
FR-05	Support opposite-side bending operations when required.	Opposite-side bending workflow with sheet-flip and regrip completion logic	Opposite-side validation test case TC-05
FR-06	Determine and apply suitable grip positions for each workpiece state.	Grip planning logic with edge-clearance, spacing, and vacuum-force feasibility checks	Grip validation test case TC-06
FR-07	Detect unsafe conditions and perform replanning or terminate execution.	Safety validation and pre-execution checking logic with replanning mechanism and controlled stop handling	Safety validation test case TC-07
NFR-01	Ensure minimal operator interaction during normal execution.	Operator interaction limited to bend-side selection and execution confirmation	Usability validation
NFR-02	Allow process parameter changes without source code modification.	Configuration reader and YAML-based single-source process configuration	Configuration validation test
NFR-03	Provide clear warning and error messages for unsafe conditions.	Warning generation and modal operator notification logic	Warning message validation
NFR-04	Ensure robustness under repeated execution cycles.	Deterministic execution loop with automatic waypoint generation and path logging	Repeated run validation test
NFR-05	Tolerate moderate pose deviations within defined tolerance limits.	Configurable pose-deviation threshold and perturbation handling logic	Perturbation validation test
SR-01	drag&bot platform shall be the virtual execution environment for robot program validation.	Runtime client connection logic for communication with the configured drag&bot environment	Platform integration check: runtime connection and command execution verified
SR-02	AnyRobot interface shall be the communication layer between planning system and robot runtime.	WebSocket-based communication layer enabling structured command exchange	Interface integration check: connection established and commands successfully executed
SR-03	All motion targets shall be defined with respect to the configured TCP.	TCP-based transformation applied in all trajectory generation and motion execution steps	TCP configuration check: consistent pose transformation verified

Table 3.3: Requirement Traceability Matrix — all requirements (FR-00 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios.

4 Development of Concepts

4.1 Purpose and Scope of Concept Development

Concept development is a fundamental stage in systematic engineering design, in which alternative solution principles are generated and evaluated prior to implementation. By comparing different approaches against defined criteria, the risk of selecting an unsuitable design solution is reduced [9].
The research gap identified in Section 2.8 highlights the absence of a systematic approach for automatic robot program generation in press brake sheet metal bending. In particular, existing methods do not adequately address changes in workpiece geometry, gripping, regripping, and pre-execution validation while minimizing manual programming effort. These challenges are translated into measurable engineering objectives and requirements in Chapter 3, including functional requirements (FR-00 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03), as summarised in Table 3.3.
To address this gap, three conceptually distinct solution approaches are developed and evaluated in this chapter. Concept A represents a structured manual programming workflow, Concept B proposes a rule-based automatic program generation approach, and Concept C explores a CAD-integrated offline programming methodology [3], [11]. These concepts are formulated based on different automation paradigms and are evaluated to identify the most suitable approach for the defined problem.
The selection of the final concept is performed using the Weighted Sum Model (WSM), a quantitative decision-making method in which evaluation criteria are assigned relative weights and each concept is scored accordingly. This ensures that the concept selection process is systematic, transparent, and aligned with the defined requirements.

4.2 Concept A – Structured Operator Workflow with Guided Teach-In Programming

4.2.1 Methodological Framework

Concept A follows a structured manual programming approach in which the operator defines robot motions and process steps explicitly, supported by a guided workflow. In this approach, robot trajectories, gripper actions, and execution sequences are created manually by the user, while the system provides a structured interface to organise the programming process.
This concept is based on conventional manual robot programming practices, as

discussed in Section 2.3, where the operator directly defines waypoints and process logic [11]. The introduction of a guided workflow improves usability by structuring the sequence of operations and reducing unstructured interaction, but it does not change the underlying requirement for manual definition of all robot motions.
Concept A may be implemented within the drag&bot environment using manually configured function blocks for robot motion, gripper control, and execution sequencing. In this case, the operator constructs the workflow by placing and configuring the required functional blocks, including motion commands and input-output operations. However, all program logic, including waypoint definition, gripping strategy, regripping operations, and process sequencing, remains user-defined. As shown in Figure 4.1, the operator manually constructs the robot program by arranging function blocks for motion commands, gripper control, and execution logic within the drag&bot environment.

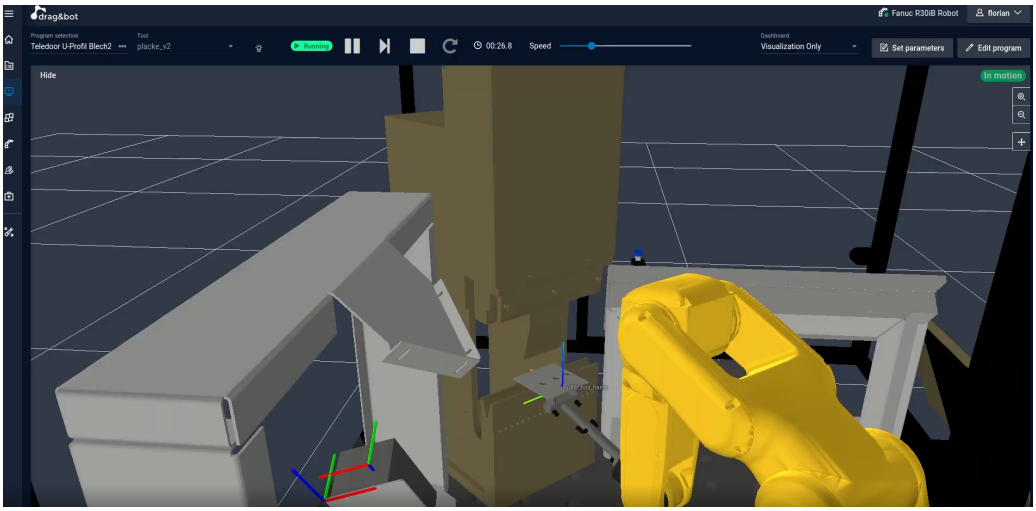


Figure 4.1: Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control.

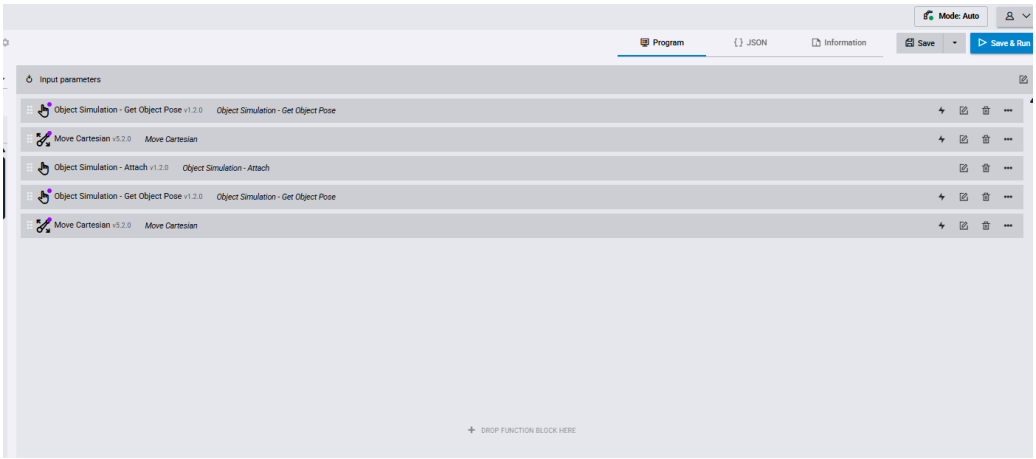


Figure 4.2: Manual robot program construction in the drag&bot environment using function blocks for motion and gripper control.

As a result, the concept does not provide automatic waypoint generation, grip point

calculation, regrip planning, or adaptive decision-making based on workpiece geometry. The role of the software environment is limited to manual program construction and execution support, rather than automated program generation.

4.2.2 System Structure

The system structure of Concept A consists of two main layers: a manual workflow construction layer and an execution layer. In the first layer, the operator defines the complete process sequence manually by specifying robot motions, gripper actions, and task order using function blocks or equivalent programming elements within the robot programming environment.
In the second layer, the execution environment interprets and executes the manually defined sequence. No intermediate planning or decision-making layer exists for automatic trajectory generation, geometric reasoning, or constraint evaluation. As a result, the system relies entirely on operator-defined logic and does not incorporate automated validation or adaptive behaviour prior to execution.

4.2.3 Critical Evaluation

Concept A is straightforward to implement and can be applied within existing robot programming environments, including drag&bot. Its operation is transparent and fully controlled by the operator, allowing direct modification of robot motions and process sequences. These characteristics make it suitable for simple and repetitive production scenarios with limited variability.
However, the limitations of this concept are significant in the context of the research objectives. The programming effort remains high, as all robot motions, handling steps, gripper actions, and process logic must be defined manually for each task. This requires substantial operator expertise in robot kinematics, tooling constraints, and process sequencing [3], [11]. As a result, the concept does not reduce manual programming effort and therefore fails to address the primary objective of this thesis.
Furthermore, the approach lacks scalability and adaptability. Changes in workpiece geometry, process sequence, or setup conditions require manual modification of the program, which limits its applicability in flexible manufacturing environments. In addition, Concept A does not provide automatic grip point determination, regripping strategies, or configuration-driven trajectory generation. Operations such as opposite-side bending and recovery from infeasible configurations remain entirely dependent on operator intervention.
Another limitation is the absence of integrated validation and decision-making. The system does not evaluate geometric feasibility, reachability, or handling constraints prior to execution. As a result, infeasible configurations are typically identified only during execution, which may lead to operational inefficiencies and potential safety risks.

Although Concept A can utilise the drag&bot environment as an execution platform, it remains a manually programmed solution and does not support automatic program generation, adaptive planning, or rule-based decision-making. Therefore, it does not satisfy the functional and non-functional requirements defined in Chapter 3, particularly those related to automation, adaptability, and validation.
For these reasons, Concept A is not suitable as the final solution for addressing the identified research gap.

4.3 Concept B – Automatic Program Generation Based on Rules with Integrated Constraint Validation

4.3.1 Methodological Framework

Unlike Concept A, which relies on manual waypoint definition, Concept B eliminates manual programming entirely by introducing a rule-based automatic program generation approach. Concept B differs fundamentally from CAD-based offline programming approaches by generating robot programs dynamically from configuration parameters and runtime validation, rather than relying on pre-defined digital models of the robot cell.
In this approach, all robot waypoints, grip positions, regripping sequences, and opposite-side handling strategies are derived automatically from geometric input parameters and a structured configuration file. Operator interaction is limited to selecting process options and confirming execution, while all motion generation and decision-making processes are handled by the system. This significantly reduces manual effort and directly addresses the primary objective of automatic program generation defined in Chapter 3. This approach directly satisfies FR-00 by enabling automatic generation of robot programs without manual intervention.
The system communicates with the robot execution environment through the AnyRobot interface provided within the drag&bot platform, using a WebSocket-based communication mechanism [19]. This architecture establishes a clear separation between the planning logic and execution environment, which is consistent with modular automation design principles [20]. As a result, the system operates independently of specific robot controller implementations while maintaining compatibility with the defined execution platform.
A key component of the methodology is the evaluation of workpiece orientation and feasibility prior to motion generation. The system retrieves workpiece pose information and evaluates admissible orientations using a structured multi-stage validation strategy. This ensures that valid handling configurations are identified regardless of the initial placement of the workpiece. Additionally, workspace constraints and admissible zones are defined through configuration parameters, allowing adaptation to different setups without modifying the underlying logic. This approach satisfies the adaptability requirement defined in NFR-02.
Grip planning is performed using a parameterised approach based on workpiece

geometry, suction cup properties, and physical constraints. Candidate grip positions are evaluated against edge clearance, spacing constraints, and vacuum holding force requirements [17]. Only physically feasible and stable grip configurations are considered for further processing, ensuring safe handling conditions prior to motion planning.
The system detects unsafe or infeasible conditions prior to execution and initiates replanning where possible, or terminates execution if recovery is not feasible, ensuring safe and reliable operation.
When a target pose cannot be reached with the current grip configuration, the system automatically initiates a regripping sequence. This sequence involves repositioning the workpiece and re-establishing a new grip configuration that restores reachability. This automated handling of regripping directly satisfies functional requirement FR-03.
The methodology also supports opposite-side bending operations through a dedicated workflow. After completing primary bending operations, the system evaluates whether opposite-side processing is required and executes a structured sheet reorientation and handling sequence. All validation steps, including orientation checking, grip feasibility, and reachability assessment, are consistently applied throughout the process [1].
Robot trajectories are generated using linear Cartesian motion commands, ensuring smooth and predictable motion execution. The use of smooth motion profiles reduces dynamic loading on the workpiece and supports stable vacuum gripping [17]. In addition, kinematic constraints such as singularity avoidance are considered during motion generation to ensure feasible and safe robot operation [11].
All motion targets are defined relative to a predefined TCP, ensuring consistency between the planning module and execution environment. This requirement directly satisfies SR-03 and guarantees that generated trajectories are aligned with the physical configuration of the robot tool.
Overall, Concept B represents a configuration-driven, rule-based methodology that enables automatic program generation, integrated validation, and adaptive handling of changing workpiece conditions. The system operates deterministically based on defined input parameters, ensuring repeatability and reproducibility of generated robot programs.

4.3.2 System Structure

The structure of Concept B consists of five layers, as defined in Section 5.2. The input and configuration layer collects all required inputs for programming. The geometry and constraints evaluation layer applies grip feasibility rules, edge-clearance checks, and zone-based orientation constraints. The planning and decision logic layer determines whether the program can be executed directly or whether regripping or workpiece flipping is required, based on reachability and orientation validity. The trajectory planning layer produces linear Cartesian waypoints with minimum-

jerk velocity profiles. The safety and validation layer applies a pre-execution pose check before the command is published to the robot runtime.
The WebSocket channel of the AnyRobot interface connects all five layers to the drag&bot virtual simulation runtime [19]. The configuration-driven approach separates all process parameters from the program logic, allowing the solution to handle new workpiece geometries by modifying configuration files. This directly satisfies non-functional requirement NFR-02, as described in Section 3.2.2.

4.3.3 Critical Evaluation

Concept B is aligned with the research gap stated in Section 2.8 for every dimension identified in the literature. It defines waypoints, plans regrips, and sequences opposite-side operations automatically from geometric parameters. A pre-execution pose validator guards against unsafe execution states before motion starts. This enables a configuration-driven architecture that can adapt to new part geometries without source code changes, satisfying FR-01, NFR-03, NFR-04, and NFR-05. Configuration-driven parameterization satisfies NFR-02. The system operates exclusively within the drag&bot runtime via the AnyRobot Interface, satisfying SR-01 and SR-02. All motion targets reference the pre-defined tool center point, satisfying SR-03. All requirements in Chapter 3 are therefore addressed by the concept design.
The primary constraints under which Concept B operates are identified in Section 3.2.3. The system assumes rectangular sheet geometry with fixed 90-degree bend-angle constraints. Integration with the KEBA robot controller hardware, which is part of the KEBA robot driver layer within the drag&bot software stack [19], is identified in the future work section and is outside the scope of this thesis. All validation has been conducted within the drag&bot virtual simulation environment, which is discussed in this section. The virtual simulation environment used for this evaluation is illustrated in Figure 4.3. The environment represents a simulated press brake cell configured with the drag&bot platform, including the robot, workpiece, and interactions required for program execution. These constraints are explicit and not indicative of design flaws but, rather, define the scope within which the design concept operates.



Figure 4.3: Virtual robotic environment used to evaluate the proposed concept (drag&bot platform).

4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation

4.4.1 Methodological Framework

Unlike Concept B, which performs rule-based program generation dynamically during execution, Concept C relies on a complete CAD-based digital model of the robot cell to generate and validate robot programs offline prior to execution.

Concept C follows a CAD integrated OLP approach, in which a digital representation of the robot, tooling, and workpiece is used to plan and simulate robot operations. In this methodology, robot programs are generated within a simulation environment based on the geometric model of the workpiece and the digital representation of the production cell. All motion planning, collision checking, and feasibility evaluation are performed prior to execution, without requiring direct interaction with the physical robot.

The OLP system evaluates bending sequences, grip positions, regripping strategies, and kinematic feasibility using the CAD model of the workpiece and environment. The generated robot program is validated within the simulation environment and then transferred to the robot controller for execution. This approach corresponds to established offline programming methods used in industrial robotics [3].

Industrial OLP tools such as ABB RobotStudio, KUKA.Sim, and FANUC ROBOGUIDE implement similar methodologies and are widely used in structured production environments [3]. The use of digital twin-based simulation enables early detection of unreachable poses, collisions, and process inconsistencies before actual execution, thereby reducing operational risks [21].

The key conceptual difference between Concept B and Concept C lies in how robot programs are generated and validated. Concept B follows a configuration-driven and rule-based approach, where robot trajectories and handling strategies are generated dynamically using input parameters and validated at runtime. In contrast,

Concept C follows a model-driven approach, where all planning and validation are performed within a CAD-based simulation environment before execution. As a result, Concept B offers higher flexibility and adaptability to changing workpiece configurations, whereas Concept C depends on accurate digital models and requires significant preparation effort for each new scenario.

4.4.2 System Structure

This concept consists of four layers. A CAD modelling layer provides digital representations of the robot, press brake, tools, and workpiece. An OLP and simulation layer uses Unified Robot Description Format (URDF) representations to plan robot programs through a built-in kinematic solver. A post-processing layer converts the generated programs into controller-specific syntax, and the robot controller layer executes the generated program. Accurate digital-twin representations of all cell components are required at the modelling layer; small modelling errors in the digital twin can propagate as execution errors that require manual reprogramming during commissioning [3], [11].

4.4.3 Critical Evaluation

Concept C offers meaningful automation advantages for structured, high-volume production environments where accurate cell models are available and OLP tool investment is justified. Simulation-based validation provides strong pre-execution assurance, and the approach is supported by mature commercial software [11], [21]. These characteristics make it the dominant paradigm in industrial robotic bending automation at the time of writing.

For the scope of this thesis, Concept C presents three fundamental disadvantages that result in its rejection. First, the implementation requires accurate CAD models of the complete robot cells, including the robot, press brake, tooling, fixtures, and workpieces — which represents a substantial engineering preparation effort that is not available within the thesis timeframe.

Second, commercial OLP systems involve high license fees while demanding expert-level instruction, both of which are unavailable within this project's constraints [9]. Besides cost, mainstream OLP platforms lack native compatibility with both the drag&bot simulation framework and the AnyRobot WebSocket system, which define the core setup targeted here [19]. Connecting such a tool would require a custom bridge to the drag&bot engine, adding implementation effort that is not justified by the scope of this thesis. Even if the first two limitations were resolved, this third limitation would disqualify Concept C, because SR-01 and SR-02 are non-negotiable constraints from the industrial partner environment.

4.5 Structural Comparison of Concepts

Design Feature	Concept A	Concept B	Concept C
Waypoint Generation Method	Manually defined by the operator for each task using teach-in or function blocks; no automation [11]	Automatically generated from sheet geometry and configuration parameters; no manual waypoint definition required [19]	Generated using CAD-based offline programming; requires model setup for each geometry variant [3]
Regrip Handling	No automation; regripping is manually planned and executed by the operator [2]	Automated through dedicated regrip logic including IO control, attach/detach operations, and sequence management [1], [2]	Regrip positions defined in simulation; execution remains manual or pre-defined without runtime adaptation [21]
Opposite-Side Bending	Manual sheet flipping and repositioning; no system support	Structured flip and handling sequence implemented through predefined logic and validated execution workflow [1]	Simulated in CAD environment; execution logic not dynamically adapted during runtime [21]
Pre-Execution Validation	Not supported; infeasible poses identified only during execution [11]	Integrated validation of orientation, reachability, and feasibility prior to execution, with support for replanning or safe termination	Validation performed in simulation environment; no runtime validation during execution [3]
Adaptability to Geometry Changes	Requires manual reprogramming for each new workpiece; low adaptability [3]	Configuration-driven approach allows parameter updates without modifying program logic	Requires modification of CAD models and re-simulation for each geometry variation [21]
Platform Compatibility	Can be executed within drag&bot as a manually constructed workflow, but does not utilise the AnyRobot Interface and therefore does not satisfy SR-02	Fully compatible with drag&bot runtime and AnyRobot Interface as defined in SR-01 and SR-02 [19]	Not compatible with drag&bot runtime or AnyRobot Interface; does not satisfy SR-01 and SR-02
Tool Centre Point (TCP) Handling	Defined manually by the operator; not enforced through system-level configuration	All motion targets defined relative to a predefined TCP, ensuring consistency with system configuration (SR-03)	Defined within CAD model; not aligned with system-specific TCP requirements
Implementation Feasibility	Easy to implement but provides no automation capability	Fully implementable within the defined software environment and thesis scope	Requires commercial OLP tools and detailed digital models; exceeds practical scope [11]
Research Gap Coverage	Does not address automation; relies on manual programming	Directly addresses the research gap through automatic program generation, regripping, and validation	Partially addresses the gap through simulation and planning, but lacks runtime adaptability

Table 4.1: Structural comparison of Concept A, Concept B, and Concept C across key design features.

Before applying the quantitative WSM, the three concepts are compared at a structural level to highlight their qualitative differences and to ensure that all requirement categories are considered. Table 4.1 presents this comparison across key design features, including automation capability, validation, adaptability, and platform compatibility.

The comparison shows that Concept B consistently addresses the core requirements of automatic program generation, including waypoint generation, regripping, opposite-side handling, and pre-execution validation. In addition, it aligns with the required software platform by integrating with the drag&bot environment through the AnyRobot Interface.

Concept A, although feasible and simple to implement, relies entirely on manual programming. While it can be executed within the drag&bot environment as a manually constructed workflow, it does not provide automated planning, validation, or configuration-driven adaptability. As a result, it does not address the primary objective of reducing programming effort.

Concept C demonstrates advantages in simulation-based validation and CAD-driven planning. However, its dependency on accurate digital models and lack of runtime adaptability limit its flexibility. Furthermore, it is not compatible with the required drag&bot and AnyRobot-based execution environment.

Overall, the structural comparison indicates that Concept B provides the most comprehensive alignment with the defined requirements, supporting the quantitative evaluation presented in Section 4.6.

4.6 Concept Decision Using the Weighted Sum Model

4.6.1 Methodology

The concept selection in this work is performed using the WSM, a structured multi-criteria decision-making method widely used in engineering design. The WSM enables quantitative comparison of alternative concepts by evaluating them against a defined set of criteria derived from the system requirements.

Each evaluation criterion is assigned a weighting factor based on its relative importance in addressing the research objectives. The concepts are then evaluated against each criterion using a consistent scoring scale from 1 to 5, where 1 indicates minimal or no satisfaction and 5 indicates a high level of satisfaction. The overall score for each concept is obtained by multiplying the assigned weight with the corresponding score and summing the results across all criteria.

This method ensures that the concept selection process remains transparent, systematic, and aligned with the functional, non-functional, and system-level requirements defined in Chapter 3. It also reduces subjectivity by providing a structured framework for comparing alternative solution approaches.

4.6.2 Evaluation Criteria and Weights

The evaluation criteria are derived directly from the functional requirements (FR-00 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03) defined in Chapter 3. The assigned weights reflect the relative importance of each criterion in addressing the identified research gap.

Programming Effort Reduction (25%) This criterion carries the highest weight as it directly addresses the primary research objective of reducing manual robot programming effort. In flexible manufacturing environments, frequent changes in part geometry require repeated program modifications, making manual programming a major limitation. Therefore, approaches that minimise or eliminate manual waypoint definition are prioritised.

Automated Regrip and Opposite-Side Handling (20%) Regripping and opposite-side bending are among the most complex and time-consuming operations in robotic sheet metal bending. These operations are often handled manually in conventional systems and significantly impact process efficiency [1], [2]. This criterion evaluates the extent to which a concept supports automated handling of these operations.

Pre-Execution Validation and Safety (20%) This criterion evaluates the ability of the system to detect infeasible or unsafe conditions prior to execution. It includes validation of reachability, orientation feasibility, and handling constraints, as well as the ability to perform corrective actions such as replanning or safe termination. This is aligned with safety and robustness requirements defined in NFR-03, NFR-04, and NFR-05.

Adaptability to Changing Workpiece Geometry (15%) This criterion assesses the flexibility of the system in handling variations in workpiece geometry without requiring significant reprogramming. Configuration-driven approaches that allow parameter modification without code changes are considered more adaptable and better suited for dynamic production environments.

Platform Compatibility (10%) This criterion evaluates whether the concept can operate within the required software environment, specifically the drag&bot runtime and the AnyRobot Interface, as defined in SR-01 and SR-02. Concepts that do not align with these platform requirements are considered less feasible for implementation within the scope of this work.

Operator Usability (10%) This criterion evaluates the level of user interaction required during operation. Systems that minimise operator involvement and reduce

the need for specialised programming knowledge are considered more user-friendly and suitable for industrial deployment, in accordance with NFR-01.

4.7 Result of the Weighted Sum Model

Table 4.2 presents the complete WSM evaluation with score justifications and weighted totals for all three concepts.

Evaluation Criterion	Wt.	Concept A — Justification	Sc.	Concept B — Justification	Sc.	Concept C — Justification	Sc.
Programming Effort Reduction	25%	All waypoints and process logic defined manually; no reduction in programming effort	1	Waypoints and process logic generated automatically from configuration parameters; minimal manual intervention required	4	CAD-based programming reduces manual teaching but requires model setup and adjustment for each variant	3
Automated Regrip and Opposite-Side Handling	20%	No automated regrip or opposite-side handling; fully operator-dependent	1	Automated regrip and opposite-side handling implemented through rule-based modules	4	Regrip and handling planned in simulation but not dynamically adapted during execution	3
Pre-Execution Validation and Safety	20%	No integrated validation; infeasible conditions detected only during execution	1	Pre-execution validation detects infeasible conditions and supports replanning or safe termination	4	Validation performed in simulation environment; no runtime validation during execution	3
Adaptability to Changing Workpiece Geometry	15%	Manual reprogramming required for each geometry change; low adaptability	1	Configuration-driven approach allows adaptation through parameter changes without code modification	4	Requires modification of CAD models and re-simulation for each new geometry	3
Platform Compatibility	10%	Can be executed within drag&bot as a manual workflow but does not utilise the AnyRobot Interface; partially satisfies platform requirements	2	Fully compatible with drag&bot runtime and AnyRobot Interface as defined in SR-01 and SR-02	4	Not compatible with drag&bot runtime or AnyRobot Interface; does not satisfy platform requirements	1
Operator Usability	10%	Requires expert knowledge and continuous manual input; high operator dependency	2	Minimal operator interaction; system handles planning and execution logic	4	Moderate usability; reduced robot expertise required but requires CAD knowledge	3
Weighted Total Score	**100%**	Concept A	**1.25**	Concept B — SELECTED ✓	**4.00**	Concept C	**2.95**

Table 4.2: Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C.

4.8 Analysis of Results and Concept Decision

The results of the WSM indicate a clear ranking among the proposed concepts. Concept B achieves the highest score (4.00), followed by Concept C (2.95), while Concept A obtains the lowest score (1.25). This reflects their relative effectiveness in satisfying the functional, non-functional, and platform requirements defined in Chapter 3.

Concept B performs strongly across all criteria. It reduces manual programming effort by generating robot programs from configuration parameters and supports automated regripping and opposite-side handling through dedicated modules. Pre-execution validation enables detection of infeasible conditions with support for re-planning or safe termination. In addition, its configuration-driven structure allows adaptation to different workpiece geometries without code modification. The concept is fully compatible with the required platform, operating within the drag&bot environment via the AnyRobot Interface.

Concept C demonstrates moderate performance through CAD-based offline programming. It enables simulation-based validation and reduces manual teaching effort; however, it depends on accurate digital models and requires reconfiguration for each geometry variation. Furthermore, it lacks runtime adaptability and is not compatible with the required drag&bot and AnyRobot environment.

Concept A shows the lowest performance. Although it can be implemented within drag&bot as a manually constructed workflow, all motions and process steps must be defined by the operator. This results in high programming effort and limited scalability. The absence of automatic planning, validation, and decision-making prevents it from addressing the defined research objectives.

Based on this evaluation, Concept B is selected as the most suitable approach. It provides the best balance between automation, adaptability, and system integration within the defined scope. The detailed implementation of Concept B is presented in Chapter 5.

5 Implementation

5.1 Introduction and Scope

The product development phase translates the selected concept into a fully realised technical system. Based on the structured concept evaluation presented in Section 4.6, Concept B was selected for implementation, as it provides a feasible and systematic solution to the research problem while satisfying the defined functional, non-functional, and software platform requirements outlined in Chapter 3. In particular, Concept B enables automatic robot program generation, integrated constraint validation, and seamless operation within the drag&bot runtime environment.

The objective of this chapter is to present the complete implementation of Concept B as an executable software system for automated press brake sheet metal bending. The system generates robot programs automatically from geometric input parameters and configuration data, eliminating manual waypoint definition and reducing operator intervention. The implementation focuses on ensuring deterministic behaviour, repeatability, and safe execution through structured validation and modular system design. The implementation is structured around four key domains. The chapter first presents the system architecture together with the communication, configuration, and operator interface structures. It then describes the development and simulation environment and the functional subsystems in execution order, including safety validation, orientation determination, grip planning, regrip handling, opposite-side bending, and trajectory generation. The chapter concludes with a Failure Mode and Effects Analysis and a requirement-to-implementation traceability mapping.

The system operates entirely within the drag&bot virtual simulation environment. Communication with the robot runtime is established through the AnyRobot Interface using an authenticated WebSocket connection, ensuring compliance with SR-01 and SR-02.

All process parameters are loaded from a central YAML Ain't Markup Language (YAML) configuration file at the beginning of each execution cycle. This enables adaptation to different workpiece geometries without modifying the source code, satisfying NFR-02.

Operator input is provided through an interactive 3D sheet bending viewer integrated into the drag&bot operator cockpit. This interface improves usability and reduces the likelihood of incorrect input selection.

Ubuntu Linux is used as the development platform due to its compatibility with ROS-based systems and stable integration with the drag&bot environment [22].

Figure 5.1 illustrates the overall system architecture and the sequence of operations from session initialization to bending execution.

The remainder of this chapter is organised as follows. Section 5.2 presents the system architecture, including the layered design, module organisation, and data flow. This is followed by the communication stack, configuration architecture, and operator interface. The simulation environment and spatial zone classification are then described. Subsequently, each functional subsystem is presented in execution order, including pre-execution safety validation, workpiece orientation strategy, grip planning, regrip handling, opposite-side bending, and trajectory generation. The chapter concludes with a Failure Mode and Effects Analysis and a requirement-to-implementation mapping. The overall execution pipeline of the system is illustrated in Figure 5.1. The workflow begins with configuration selection and parameter initialisation, followed by pre-execution safety validation to ensure that all operating conditions are within acceptable limits. Based on the detected sheet pose, a valid pickup orientation is determined, after which feasible grip positions are computed using constraint-based evaluation. If a direct execution is not possible, the system automatically applies regrip strategies to achieve a reachable configuration. The validated task sequence is then converted into motion commands and executed by the robot. The process iterates over all required bends and includes additional handling for opposite-side operations before concluding with the final unloading stage. This structured workflow ensures deterministic execution and consistent integration of all functional subsystems within the automated bending process.

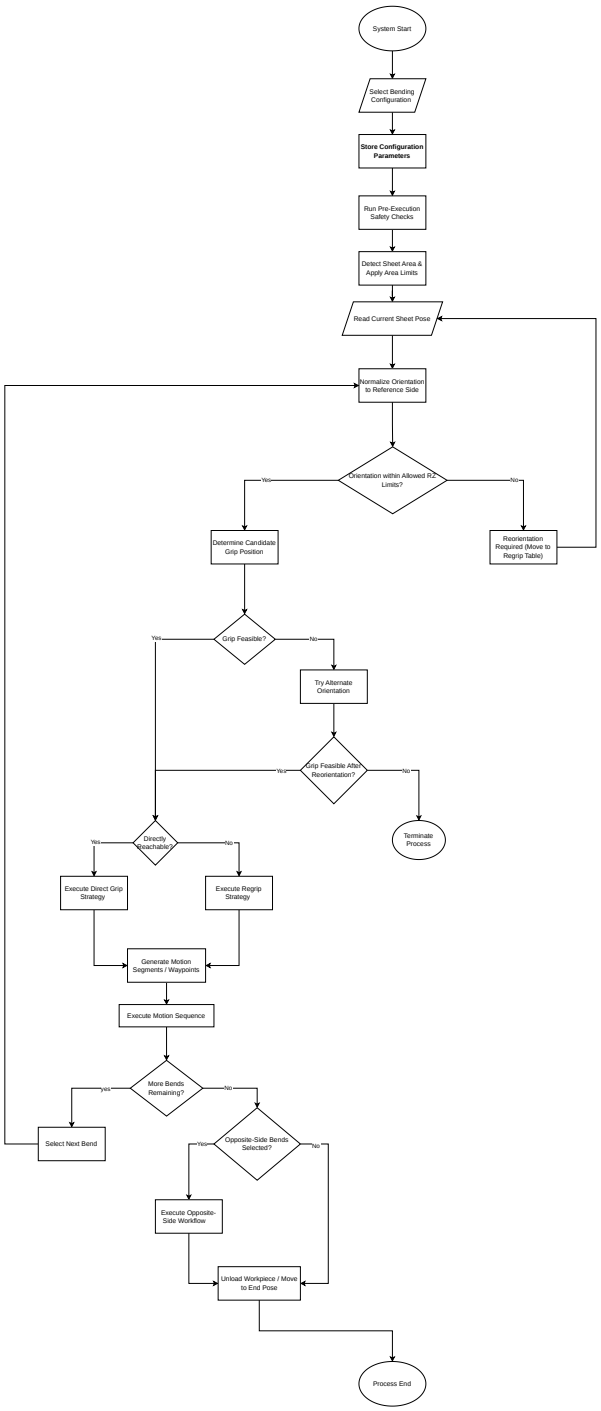


Figure 5.1: Overall system workflow and execution pipeline.

5.2 System Architecture

5.2.1 Layered Architecture Design and Data Flow

The system adopts a layered architecture in which each module depends exclusively on the module in the layer directly beneath it. This architectural pattern is selected because the press brake automation context imposes three key constraints that cannot be satisfied by an unstructured design: independent testability of con-

straint evaluation, maintainability and safe replaceability of components, and strict enforcement of safety ordering. Testability is achieved by enabling modules to be evaluated using synthetic pose data and YAML-based configuration inputs, without requiring an active WebSocket connection. This isolation is essential for verifying orientation classification and grip feasibility computations, which cannot be reliably validated from live system behaviour alone.

Replaceability is ensured through modular separation, where modifications to the grip planning algorithm are confined to the constraint evaluation layer. The planning and motion layers remain independent of these modules, supporting maintainability and satisfying NFR-02.

Safety ordering is enforced structurally by implementing pre-execution validation in a dedicated layer that is executed prior to any motion generation. This guarantees that safety checks cannot be bypassed or reordered, satisfying FR-07 and NFR-03.

Five distinct layers are defined within the system architecture, each with a clearly assigned responsibility. Layer 1 (Input and Configuration) is responsible for acquiring all required parameters from the input file, along with the operator's bend-side selection, prior to the establishment of any WebSocket connection. Layer 2 (Geometry and Constraint Evaluation) performs the computation of geometric and physical constraints based on input pose data, while remaining strictly limited to evaluation tasks and producing no motion commands. Layer 3 (Planning and Decision Logic) determines the overall execution strategy and coordinates the interaction between Layers 2 and 4, without directly generating motion instructions. Layer 4 (Motion Generation and Communication) serves as the exclusive interface for robot communication, generating Linear motion command (LIN) and Point-to-point motion command (PTP) waypoints, converting them into JavaScript Object Notation (JSON) format, and transmitting them via an authenticated WebSocket connection. Layer 5 (Safety and Validation) is implemented as an independent top-level layer that is executed prior to any motion generation and enforces mandatory validation and operator confirmation, preventing further execution until all safety conditions are satisfied. The complete architectural structure is illustrated in Figure 5.2.

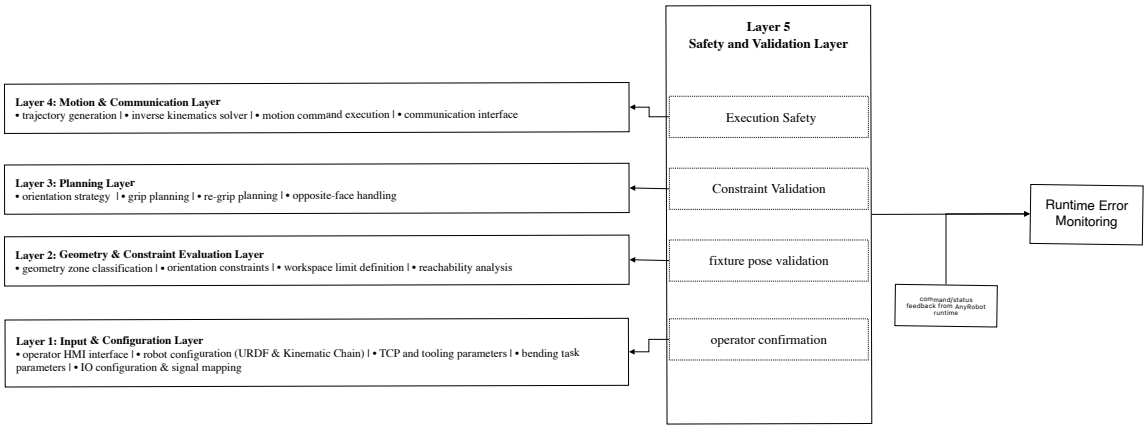


Figure 5.2: Global system architecture for validation workflow.

The progression of data throughout a bending cycle follows a structured transformation across the system layers. Initially, sheet geometry, bend-side selection, and tool parameters are provided as input. These inputs are processed within the constraint evaluation layer, where raw pose data is transformed into a validated pickup orientation and feasible grip configuration. Based on this information, an execution strategy is determined and translated into motion commands, which are encoded in JSON format and transmitted to the runtime environment.
The reliability of the overall process depends on several critical transformation stages. Any error introduced at these stages propagates through subsequent computations and affects execution outcomes. In particular, accurate orientation determination is essential for successful workpiece acquisition, while grip feasibility evaluation ensures stable handling during transport. During trajectory generation, a balance must be maintained between path accuracy and controller limitations, requiring controlled reduction of trajectory points while preserving motion characteristics.

5.2.2 Module Overview and Dependency Discipline

The implementation consists of software modules organized across the five system layers. Table 5.1 summarizes each module in terms of its layer assignment, functional role, and primary responsibility. Each module is defined with a single primary responsibility to ensure clear fault attribution during failure analysis. Assigning multiple responsibilities to a single module would make it difficult to associate a specific failure mode with a well-defined functional unit.

No.	Layer	Module	Primary Responsibility
1	L1	Session launcher	Coordinates YAML loading, 3D viewer initialisation, and session start-up before runtime connection.
2	L1	Configuration reader	Loads and validates configuration data and provides a consistent parameter source for the remaining modules.
3	L5	Safety checker	Enforces operator confirmation and safety validation before execution begins.
4	L2	Zone classifier	Classifies tracker data into spatial zones and determines admissible orientation strategies.
5	L2	Orientation selector	Determines the pickup orientation on the basis of geometric evaluation of the workpiece.
6	L2	Face orientation converter	Converts tracker pose data into a reference-face representation using rotation transformations.
7	L2	Grip position planner	Computes feasible grip positions while considering edge clearance and holding constraints.
8	L2	Pickup orientation validator	Verifies tool alignment and applies corrective transformations where required.
9	L4	Trajectory generator	Generates waypoint sequences with path refinement, sampling control, and singularity avoidance.
10	L4	Regrip motion executor	Executes regrip operations, including transport, release, and re-acquisition.
11	L3	Primary bending orchestrator	Coordinates execution flow across orientation, planning, and motion modules.
12	L3	Regrip sequencer	Determines transition sequences between grip configurations.
13	L3	Opposite-side runner	Evaluates and executes alternative orientation strategies when required.
14	L3	Multi-stage regrip loop	Handles iterative repositioning for complex bending sequences.
15	L3	Post-bend unload sequencer	Controls unloading of completed workpieces to the designated buffer location.

No.	Layer	Module	Primary Responsibility
16	L4	Waypoint data structure	Encapsulates robot motion parameters, including pose, velocity, and blending information.
17	L4	Digital IO handler	Manages vacuum control and safety-related input/output signals.
18	L4	WebSocket client	Handles authentication, session management, and message transmission to the runtime interface.
19	L4	Command type definitions	Defines motion categories, coordinate conventions, and JSON message formats.

Table 5.1: Software modules across system layers and their primary responsibilities.

Three structural properties ensure that the architectural constraints are consistently enforced across the system. First, the WebSocket client and the command type definitions together constitute the exclusive communication layer. No module outside Layer 4 constructs WebSocket messages or performs direct network operations. As a result, authentication handling and connection recovery are implemented centrally and remain consistent across all runtime interactions. Second, the safety checker is positioned within Layer 5, which is not imported by any lower layer. This separation prevents planning or execution modules from invoking safety validation during runtime. Instead, safety checks are enforced strictly at the beginning of a session, ensuring that they cannot be bypassed or reordered during execution. Third, the trajectory generator serves as the sole interface for dispatching motion commands to the runtime. All waypoint sequences are routed through this module, ensuring that workspace constraints and singularity checks are consistently applied before execution.

5.2.3 Communication Stack and Authentication Interface Specification

All runtime communication passes through the AnyRobot Interface, as it is the only supported external Application Programming Interface (API) for drag&bot. Direct ROS client libraries are excluded because they would couple the planning software to a specific ROS version, violating NFR-02 [19] [22]. The AnyRobot Interface translates JSON over a persistent WebSocket channel into native ROS operations using the RosBridge v2 protocol, enabling the entire planning software to run in standard Python 3 without a ROS installation on the planning machine.
WebSocket is selected over Hypertext Transfer Protocol (HTTP) due to its ability to maintain a persistent connection that supports continuous data exchange. Unlike HTTP, which follows a request–response model and requires a new connection for each interaction, WebSocket enables uninterrupted communication over a single

channel. In applications involving robotic motion execution, where a large number of trajectory points must be transmitted sequentially, repeated connection overhead can introduce noticeable delays. These delays may accumulate and lead to interruptions in command streaming, potentially affecting smooth robot operation. By using a persistent communication mechanism, WebSocket reduces transmission overhead and ensures more stable and efficient delivery of motion commands, making it well suited for real-time robotic control scenarios [23].

Figure 5.3 presents an overview of the system communication architecture. The planning software establishes a persistent WebSocket connection to the robot controller via a defined network endpoint (e.g., `ws://192.168.x.x:9091`). Communication is organized through a set of dedicated interfaces, each responsible for a specific function within the system.

WebSocket-based middleware is widely used in robotics to enable platform-independent communication between external applications and ROS-based systems [24]. Such approaches rely on a persistent socket connection and lightweight message serialization to support continuous data exchange required for real-time operation.

In the implemented system, the Ros Bridge v2 interface is used to transmit motion commands, while the `rosauth` service ensures secure authentication using the Secure Hash Algorithm 512-bit (SHA-512) hashing algorithm. Motion trajectories are transmitted in batches via the `/command_list` interface, and real-time object information is retrieved through `/dnb_object_manager/list_objects`. Gripper control is performed using `/robot/set_digital_ios`, whereas object attachment and detachment are handled through the corresponding manager interfaces. User interactions are managed via the `human_interaction/request_hi` interface.

The YAML configuration is not part of the runtime communication. It is loaded once during system initialization and shared across the relevant modules, ensuring that missing or incorrect parameters are detected early before execution begins.

On connection establishment, the WebSocket client performs the rosauth SHA-512 authentication handshake, because the drag&bot WebSocket port is accessible to any process on the local network and an unauthenticated endpoint would allow arbitrary motion command injection. The Message Authentication Code (MAC) is computed by hashing with SHA-512 concatenation of the shared runtime secret, the interface identifier, destination, a fixed nonce, session timestamps, and the access level string.

The authentication token is computed based on the procedure defined in the drag&bot AnyRobot Interface specification [19]:

$$\text{MAC} = \text{SHA-512}(\text{secret} \parallel \text{id_string} \parallel \text{dest} \parallel \text{nonce} \parallel \text{t} \parallel \text{level}) \tag{5.1}$$

SHA-512 is selected due to its strong cryptographic properties, as its 512-bit output makes pre-image and collision attacks computationally infeasible within the operational context of this application. The shared secret is stored as an environment

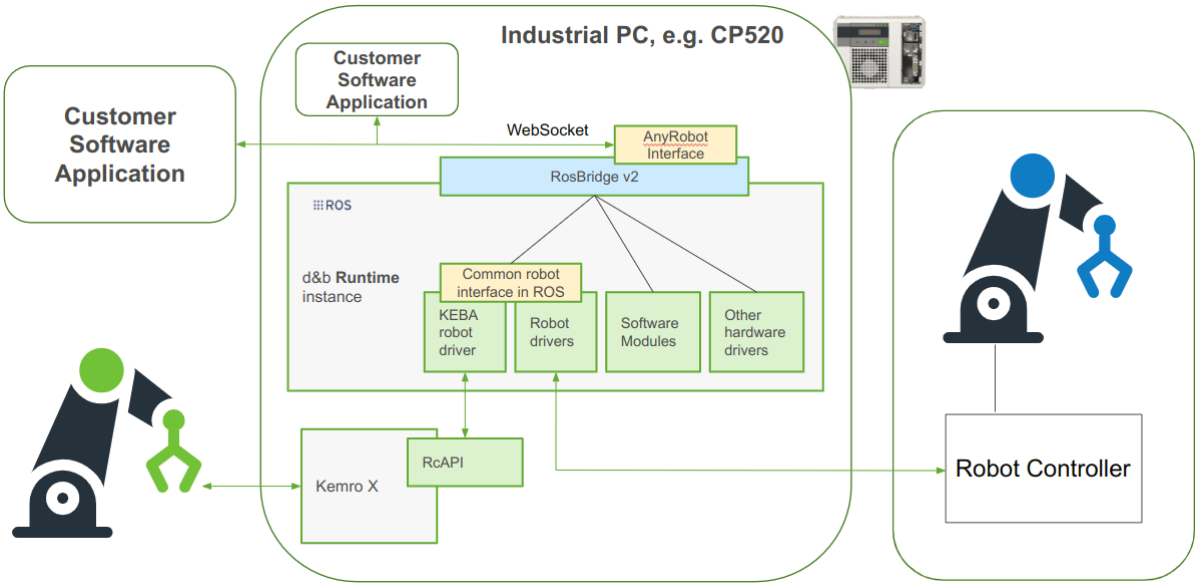


Figure 5.3: Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [19]).

variable to prevent exposure to version control systems. The concatenation order is strictly defined by the AnyRobot interface specification and must be followed precisely. Any deviation from this order results in a mismatch of the generated hash, causing the runtime system to reject the connection.

Listing 5.1 presents an illustrative Python implementation of the authentication token computation described in Equation 5.1, following the drag&bot AnyRobot Interface specification [19].

```python
import hashlib
import os

secret = os.getenv('RUNTIME_SECRET')

id_string = 'library'
dest = 'interface'
nonce = 'example_nonce'
timestamp = 0
role = 'user'

raw = secret + id_string + dest + nonce + str(timestamp) + role

mac = hashlib.sha512(raw.encode()).hexdigest()

auth_msg = {
    'operation': 'auth',
    'mac': mac,
    'nonce': nonce,
    'timestamp': timestamp,
```

```
    'role': role
}
```

Listing 5.1: Illustrative Python implementation of SHA-512-based authentication token computation following the drag&bot AnyRobot Interface specification [19].

Figure 5.4 shows the robot control panel used for system interaction and monitoring.

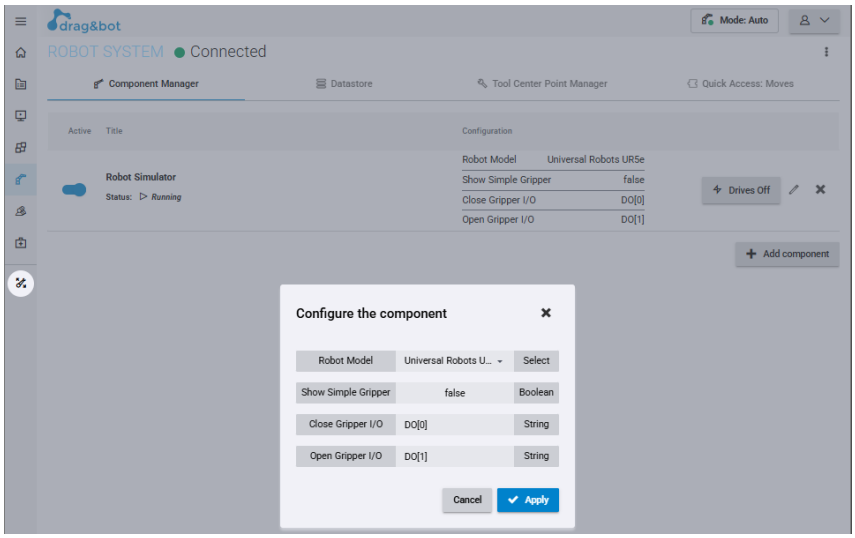


Figure 5.4: Robot control panel interface used for system operation and monitoring.

5.2.4 Configuration Architecture and Data File Structure

The YAML configuration file serves as the central entry point for all process-related parameters and provides a flexible mechanism for adapting the system to different workpieces without requiring modifications to the underlying source code, thereby fulfilling NFR-02. At the beginning of each session, the file is parsed once, and the resulting configuration data is stored as a dictionary that is shared across all modules in Layers 2 to 4. The structure of the file is organized into six clearly defined top-level sections, each corresponding to a specific functional aspect of the system architecture, which supports clarity and maintainability. YAML is selected over JSON primarily due to its support for inline comments. This allows engineers to document the meaning and physical relevance of individual parameters directly within the configuration file, eliminating the need to consult external documentation. To ensure robustness and security, the configuration is loaded using a safe parsing method, which prevents the execution of unintended code in the event that the file is modified or corrupted. A key design decision concerns the handling of spatial consistency during the regrip process. Rather than defining approach and placement poses independently, all such poses are derived relative to a single reference pose defined for the orientation table. For example, the approach position is calculated by applying a fixed vertical offset to the placement pose. This design ensures that any change in the position of the orientation table requires only a single modification within the configuration file. In contrast, storing these poses independently could

introduce inconsistencies: if the table position is updated but the corresponding approach position is not, the system may execute motions based on outdated spatial data. Such inconsistencies are particularly problematic because they may not be detected during configuration validation, as each parameter remains individually valid despite being collectively incorrect.
Listing 5.2 presents a generalised YAML configuration illustrating the key process, motion, and safety parameters without exposing system-specific or confidential information.

```yaml
# Robot configuration (generalised)
robot:
  host: <robot_ip_address>
  port: <port_number>
  speed_mps: 0.5
  accel_percent: 20
  blend_percent: 15
  io_group: DO
  io_pin: 1
# Process and sheet definition
process:
  sheet:
    length_m: 0.4
    width_m: 0.3
    thickness_m: 0.005
    mass_kg: 2.5

  grip:
    cup_count: 2
    min_edge_clear_m: 0.05
    min_spacing_m: 0.2
```

Listing 5.2: Generalised YAML configuration illustrating robot parameters, process definition, and motion and safety constraints.

5.2.5 Operator Interface and Interactive Three-Dimensional Sheet Bending Viewer

Operator input for each execution cycle is provided through an interactive three-dimensional sheet bending viewer developed in this work (hereafter referred to as the *3D viewer*), rather than a conventional text-based interface. This design decision is primarily motivated by the spatial nature of bend side selection. By presenting the workpiece directly as a 3D model, the operator can identify relevant surfaces and orientations more intuitively than by relying on abstract side labels. In practice, this reduces the likelihood of incorrect selections while keeping the interaction simple, as the operator performs only visually guided inputs and the system handles the underlying computations.

The 3D viewer is implemented as a web-based application using Plotly Dash and renders the workpiece as an interactive mesh derived from the corresponding Standard Tessellation Language (STL) file. It operates as a local service and is integrated into the drag&bot Operator Cockpit through the Custom URL panel mechanism. A new panel of type *Custom URL* is added to the dashboard via the dashboard configuration interface, where the local viewer address is specified as the target. Once configured, the viewer is rendered as an embedded panel within the cockpit, allowing the operator to interact with the 3D mesh and associated controls without leaving the drag&bot environment. The configuration is stored as part of the dashboard definition and remains available across sessions, ensuring a consistent setup for repeated operation.

This integration ensures that configuration, monitoring, and execution are performed within a single interface, eliminating unnecessary context switching and providing a more coherent workflow from the operator's perspective. The overall system setup used during development and validation, including the robot, press brake, and orientation table, is shown in Figure 5.5.

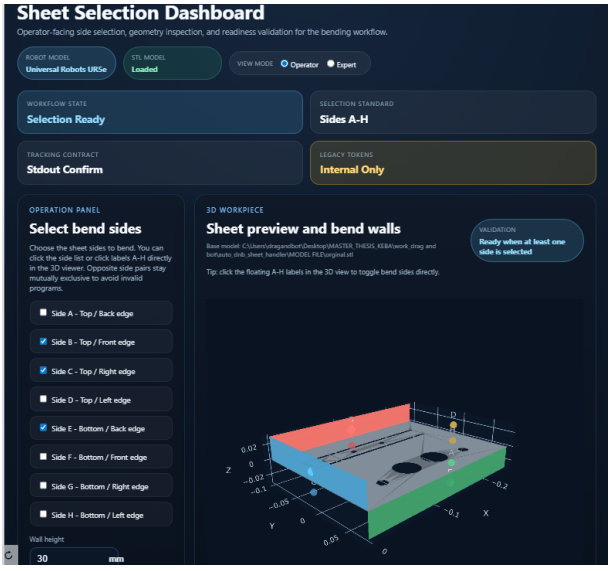


Figure 5.5: Operator workflow illustrating the sequential execution stages. Each stage must be completed before the next begins. All operator interactions occur within the drag&bot environment. *Source: own work.*

To accommodate different levels of operator expertise, the interface is divided into two modes. In basic mode, the operator selects from a predefined set of bending sides and specifies the required wall height. The system automatically prevents invalid combinations, particularly those that would require additional operations such as regripping, which are intentionally excluded at this level to simplify interaction. In contrast, expert mode exposes additional parameters, including geometric constraints, tool configuration, and pose definitions. However, changes made in this mode are not stored automatically and must be explicitly transferred to the configuration file, ensuring that persistent modifications are introduced in a controlled manner.

In parallel with user input, the system performs a geometric analysis of the imported mesh to identify internal features such as holes. These features are detected based on their orientation and their spatial relationship to the outer boundary of the sheet, and are subsequently classified according to their geometric characteristics. The resulting feature information is used during grip planning to exclude unsuitable regions from consideration. The workpiece model used for this analysis is illustrated in Figure 5.6. Additionally, the workpiece mass is estimated using a standard material density, providing a conservative reference for evaluating whether a given gripping configuration is physically feasible.

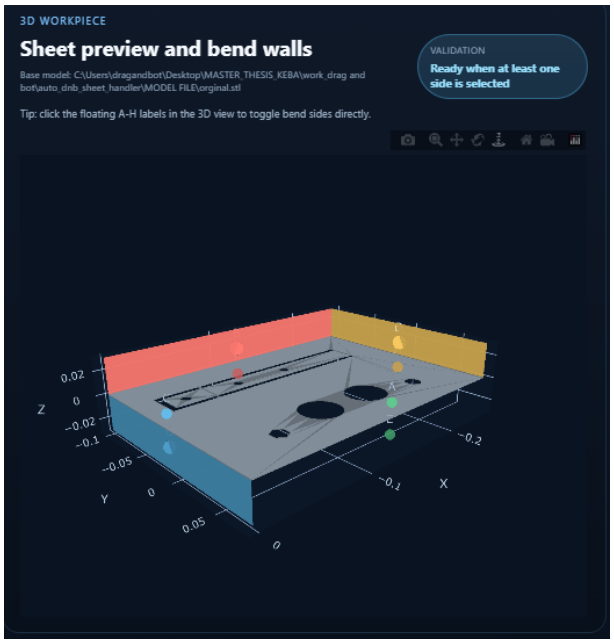


Figure 5.6: Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work.

Once the operator confirms the selection, the defined parameters are transmitted to the planning module in a structured format. The viewer then terminates after a short delay. This behaviour is intentional, as it prevents further interaction while the robot is executing a trajectory, thereby avoiding inconsistencies that could arise if configuration changes were introduced during active motion.
The complete sequence of operator interaction and system processing is summarised in Figure 5.7, which illustrates the progression from initial configuration through planning, strategy selection, and final execution.

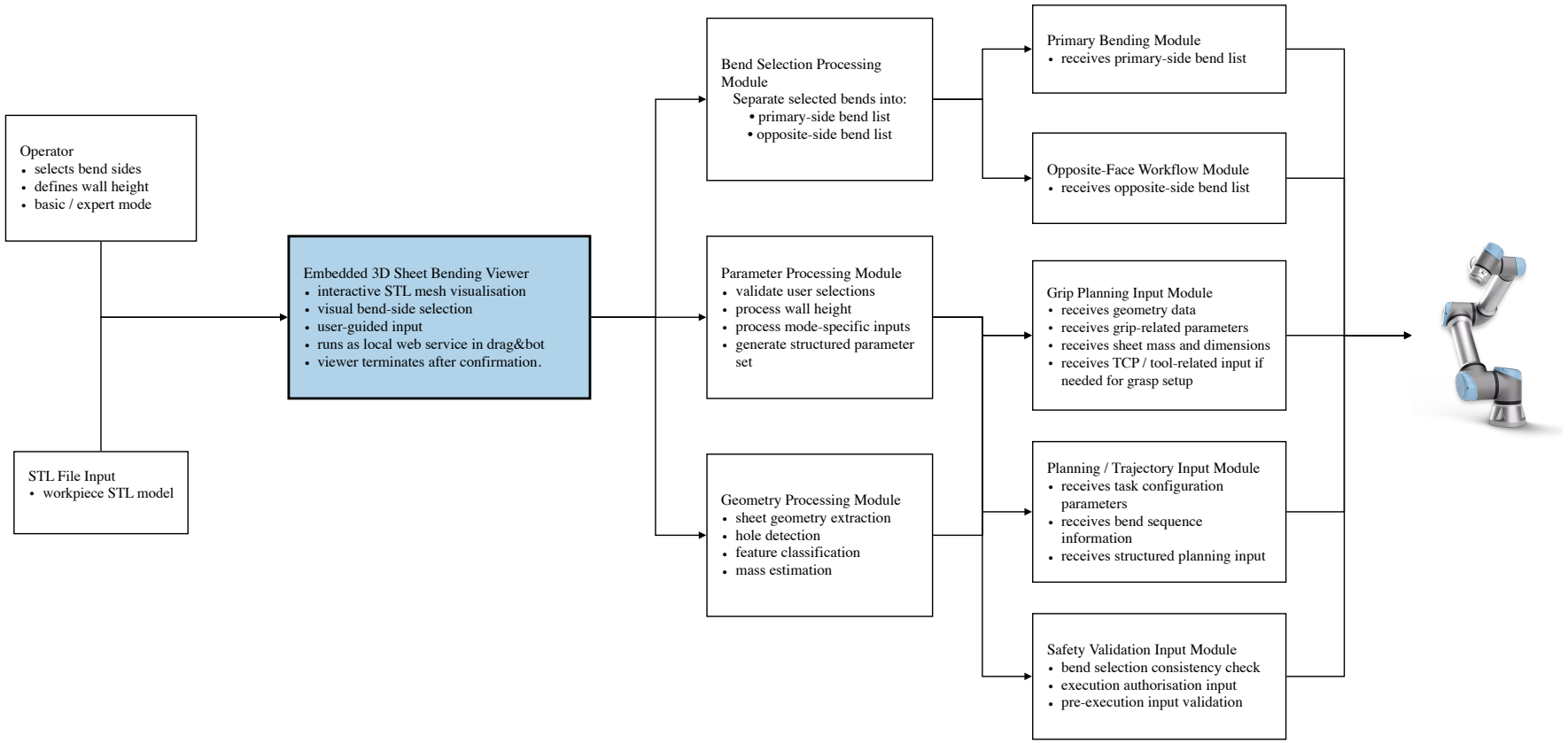


Figure 5.7: Operator workflow. Every gate must be passed before the next begins. All operator interactions occur within the drag&bot environment. *Source: own work.*

5.3 Development and Simulation Environment

With the architectural framework established in Section 5.2, a controlled environment is required to execute, test, and validate the developed system prior to deployment on physical hardware. This section presents the development and simulation environment used in this work, including the drag&bot platform, the robot model and tool configuration, and the spatial zone classification strategy used to structure the workspace.

5.3.1 Simulation Platform, Robot Model, and Tool Centre Point Configuration

The drag&bot platform is used as the primary environment for system development, integration, and validation. It provides a virtual robot controller together with the AnyRobot WebSocket Interface, enabling the planning software to interact with the robot runtime without requiring direct access to physical equipment. This supports safe testing, repeatability, and efficient iteration during development. The use of virtualised environments is consistent with current industrial trends in digital manufacturing and digital twin-based validation [19], [25].
All modules are implemented in Python and executed within an Ubuntu-based environment alongside the drag&bot runtime. Communication is established through a single authenticated WebSocket connection (e.g., `ws://<local-ip>:9091`), which is used for motion execution, IO control, object management, and operator interaction. This unified communication structure ensures consistent behaviour across all subsystems.
All session-specific parameters are initialised at the beginning of execution and explicitly passed between modules. This avoids hidden dependencies and ensures that all components operate on consistent data, supporting modularity and maintainability.
Within this environment, the Universal Robots Universal Robots 5e (robot model) (UR5e) is used as the manipulator model. The robot provides six degrees of freedom, a reach of approximately 850 mm, and a payload capacity of 5 kg, which are sufficient for handling the sheet metal workpieces considered in this work. The availability of a validated kinematic model within the drag&bot platform allows all generated motion targets to be evaluated for reachability and feasibility during simulation [19], [26].
The TCP used for all motion targets is defined in the drag&bot tool management system as `project_3_tcp`, in accordance with requirement SR-03. The TCP is specified using the ZYX Euler convention with offsets relative to the robot flange: $x = 0.000$ m, $y = 0.034$ m, $z = 0.194$ m, $RZ = -90.0°$, $RY = -75.0°$, and $RX = 0.0°$. These parameters ensure that all generated poses remain consistent with the physical gripper configuration. The TCP identifier is provided through the operator input, allowing future adjustments without modification of the underlying source code.

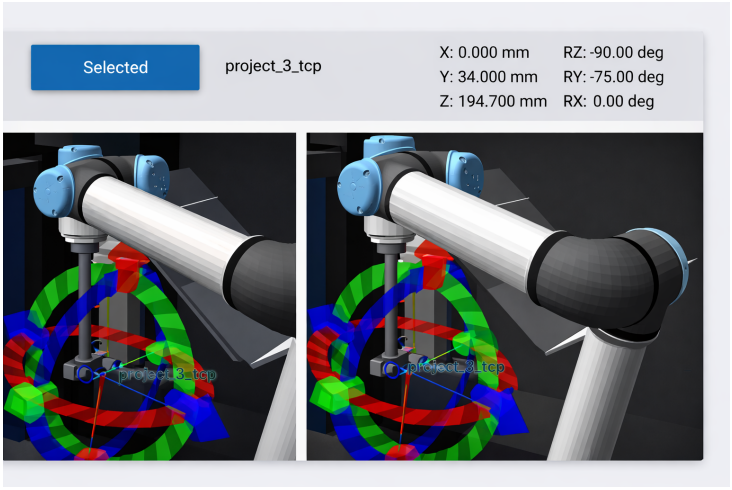


Figure 5.8: Tool Centre Point (TCP) configuration of the UR5e robot used for motion planning. *Source: own work.*

5.3.2 Virtual Cell Layout and Spatial Zone Classification

The simulated robotic cell within the drag&bot environment includes the robot, press brake, orientation table, and a tracked sheet object, which serves as the reference for all geometric computations. The spatial configuration of these elements reflects the intended physical setup, ensuring that simulation results are representative of real-world operation.

At the beginning of each execution cycle, the system retrieves the six-degree-of-freedom pose of the sheet from the object tracking service. The pose is expressed in the robot world coordinate frame, where the origin is located at the robot base. Accurate pose acquisition is a fundamental requirement in robotic manipulation systems and is widely addressed in modern robotics literature [11]. Based on this information, the position of the sheet centroid in the XY plane is used to classify the workpiece into one of four spatial zones.

The workspace is partitioned into four regions relative to the robot base. A centroid with negative X and positive Y corresponds to Zone 1 (rear-left), while negative X and negative Y defines Zone 2 (front-left). Positive X and positive Y correspond to Zone 3 (rear-right), and positive X with negative Y defines Zone 4 (front-right). The classification is based on threshold values of ± 10 mm in the X direction and ± 50 mm in the Y direction. These thresholds account for measurement uncertainty and reflect the physical layout of the robotic workspace [11].

Zones 3 and 4 lie within the primary reachable workspace of the robot and therefore allow direct execution of pickup and bending operations. In contrast, Zones 1 and 2 correspond to regions with limited reachability. In these cases, the system automatically initiates a repositioning step, moving the workpiece to an intermediate configuration within the reachable workspace before continuing execution. This ensures that all subsequent operations remain within kinematic limits without requiring operator intervention.

If the sheet centroid lies within the dead-band region defined by $|x| < 10$ mm or

$|y| < 50$ mm, the classification is considered unresolved. This occurs near the coordinate axes, where small variations in tracking data may lead to inconsistent zone assignment. To prevent incorrect execution, the system terminates the current session and prompts the operator to reposition the workpiece before restarting.

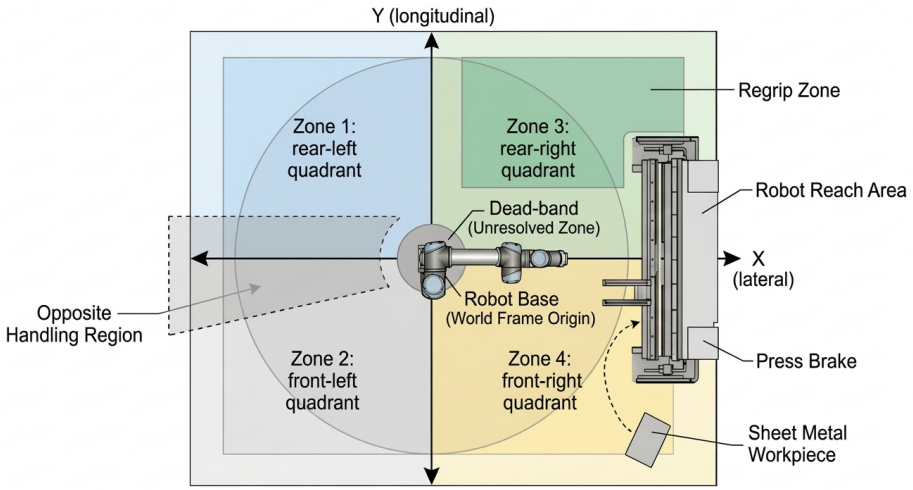


Figure 5.9: Spatial zone classification of the robot workspace based on the sheet centroid position. *Source: own work.*

Having defined the simulation environment, robot model, TCP configuration, and workspace segmentation strategy, the following sections describe the functional subsystems in the order of execution within a bending cycle, beginning with safety validation, followed by orientation determination, grip planning, and trajectory generation.

Algorithm 1 Spatial zone classification based on sheet centroid position.

Retrieve sheet centroid coordinates (x, y) in the robot world frame
if $|x| < x_{\text{th}}$ **or** $|y| < y_{\text{th}}$ **then**
    Return *unresolved zone*
else if $x < 0$ **and** $y > 0$ **then**
    Assign Zone 1
else if $x < 0$ **and** $y < 0$ **then**
    Assign Zone 2
else if $x > 0$ **and** $y > 0$ **then**
    Assign Zone 3
else
    Assign Zone 4
end if

Algorithm 1 summarises the classification logic in an abstract form. The method relies on the sign of the centroid coordinates together with a threshold-based dead-band region to ensure stable classification under measurement uncertainty.

5.4 Pre-Execution Safety Validation

Safety validation is performed prior to motion generation to ensure that unsafe conditions and infeasible configurations are identified before robot execution, in accordance with NFR-03, NFR-04, and NFR-05 [11]. The system incorporates three complementary mechanisms: a fixture pose deviation check, a blocking operator confirmation dialog, and a runtime motion error monitoring module. In addition, all trajectory waypoints are generated and verified using robot kinematics to ensure reachability and feasibility before execution, thereby preventing invalid motion commands from being issued [11].

5.4.1 Fixture Pose Deviation Validation

The fixture pose deviation check retrieves the current poses of critical fixtures from the drag&bot object manager and compares them with reference poses obtained during initial cell calibration. The monitored elements include the primary sheet object at the press brake, the finished-part placement positions on the regrip table, and the placement and attachment references at the flip station. These fixtures are treated as fixed references within the planning system, as all grip positions, trajectory waypoints, and placement operations are computed under the assumption that their positions remain unchanged. Any deviation from these reference poses can therefore result in motion targets that are geometrically valid but physically incorrect.

The deviation check is applied uniformly across all spatial zones. Each fixture is evaluated using both positional and orientation criteria. The positional deviation is computed as:

$$\Delta_{pos} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{5.2}$$

and the orientation deviation is defined as:

$$\Delta_{ang} = \max\left(|\Delta RZ|, |\Delta RY|, |\Delta RX|\right). \tag{5.3}$$

A fixture is considered out of tolerance when Δ_{pos} exceeds 5 mm or Δ_{ang} exceeds 0.10 rad (5.7°). The positional threshold is selected to remain below the minimum clearance used in grip planning. This ensures that fixture misalignment does not produce misleadingly valid grip configurations. The angular threshold corresponds to the minimum deviation that would affect the accuracy of vacuum cup contact on the sheet surface.

The primary sheet object is evaluated using the same criteria as other fixtures. Since the planning system relies on the current sheet pose for all geometric computations, any deviation may lead to incorrect motion generation and is therefore treated as a configuration error.

If one or more fixtures exceed the defined thresholds, the system generates a

structured warning listing all affected fixtures together with their measured positional and angular deviations. This warning is presented through a blocking human interaction (HI) dialog titled *Fixture position warning*, allowing the operator to either proceed with explicit acknowledgement or cancel execution for further inspection. The operator's decision is recorded, ensuring that responsibility for accepting out-of-tolerance conditions remains with the human operator.
If a fixture is not detected in the object manager response, it is treated as missing and automatically flagged as a validation failure.

5.4.2 Operator Authorization and Runtime Error Monitoring

The operator confirmation dialog is positioned as the first step in the planning orchestrator startup sequence, preceding all subsequent processes including fixture validation, visualization, and motion execution. The dialog enforces a mandatory decision: either initiate the automated workflow or cancel execution. The orchestrator does not proceed until a response is received from the HI service, ensuring that execution cannot begin without explicit operator authorisation. If the operator selects cancel, the process terminates without issuing any command to the robot runtime.
In addition to pre-execution validation, runtime supervision is implemented through a motion error monitoring module operating as a background subprocess. This component subscribes to the `/command_result` ROS topic via an independent WebSocket connection, enabling continuous monitoring of execution outcomes without interrupting the main command flow. When a non-zero result code is detected, indicating a motion execution failure, the system immediately triggers a modal HI notification to inform the operator.
The monitoring module does not directly interrupt the execution pipeline. Instead, it provides immediate feedback through the HI interface and requires operator acknowledgement before further interaction is permitted. This ensures that execution anomalies are not overlooked while maintaining operator authority over subsequent decisions.
Together, these mechanisms establish a layered safety structure. Pre-execution validation ensures safe initial conditions, while runtime monitoring provides immediate feedback on execution failures. Both mechanisms utilise the drag&bot HI service, ensuring a consistent and controlled interface for safety-related interactions. With these safeguards in place, the subsequent section addresses the workpiece orientation strategy.

5.5 Workpiece Orientation Strategy

Following pre-execution safety validation, the first geometric step in each bending cycle is the determination of a valid pickup orientation for the sheet. This is required because the sheet may be placed in arbitrary orientations within the

workspace, while only a subset of these orientations is kinematically feasible for the target bending operation. The orientation strategy evaluates the tracked sheet pose, applies a structured selection process, and produces a validated pickup pose for subsequent grip planning.

5.5.1 Zone-Based Rotation Admissibility

The admissible range of sheet rotation about the Z-axis is defined by a zone-dependent window $[RZ_{\text{min}}, RZ_{\text{max}}]$. This constraint reflects the limited rotational capability of the robot end-effector within different regions of the workspace [11]. Since the reachable orientation range varies with the spatial position of the sheet, zone classification is performed prior to orientation evaluation. The admissibility limits are retrieved from the configuration to ensure that only kinematically feasible orientations are considered. This prevents the selection of geometrically valid but unreachable poses.

5.5.2 Pickup Orientation Validation

Before applying the selection strategy, the tracked pose is validated to ensure that the gripper approaches the sheet from above. This is evaluated using the condition $\cos(RX) \cdot \cos(RY)$. If the value is below -0.10, the tool Z-axis is oriented downward and no modification is required. Otherwise, a corrective transformation is applied:

$$RZ' = \text{wrap}(RZ + 180^\circ), \quad RY' = -RY, \quad RX' = \text{wrap}(RX + 180^\circ) \tag{5.4}$$

This transformation ensures proper tool alignment with the sheet surface without modifying the pickup position.

5.5.3 Three-Level Orientation Selection Strategy

To ensure that a valid pickup orientation is always obtained, a three-level selection strategy is applied. The levels are evaluated sequentially, and execution terminates as soon as a valid orientation satisfying admissibility constraints is identified. This guarantees robustness while prioritising geometrically consistent and kinematically feasible configurations.
The workpiece is defined using four reference faces established during system commissioning, each associated with a fixed angular offset about the sheet's local vertical axis (0°, 180°, $+90^\circ$, -90°). These offsets allow the system to normalise arbitrary orientations into a consistent reference frame.

Level 1: Reference-Face Equivalent Orientation At the highest priority level, the tracked orientation is converted into a reference-face equivalent by removing the face-specific offset:

$$R_{\text{new}} = R_{\text{raw}} \cdot R_z(-d_{\text{face}}) \tag{5.5}$$

The resulting rotation matrix is decomposed into ZYX Euler angles [11], [27]:

$$R = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma) \tag{5.6}$$

$$RZ = \text{atan2}(R_{10}, R_{00}), \quad RY = \text{arcsin}(-R_{20}), \quad RX = \text{atan2}(R_{21}, R_{22}) \tag{5.7}$$

If the computed RZ lies within the admissibility window, this orientation is selected. This level is preferred as it ensures consistent reference alignment and reduces trajectory complexity.

Level 2: Detected-Pose Evaluation If Level 1 fails, the raw tracked orientation is evaluated directly. This accounts for cases where the initial placement is already feasible and avoids unnecessary transformations. If the raw RZ satisfies admissibility constraints, it is selected.

Level 3: Inverted-Pose Fallback If neither Level 1 nor Level 2 satisfies the admissibility condition, a fallback orientation is generated:

$$RZ' = \text{wrap}(RZ \pm 180^\circ), \quad RY' = -RY, \quad RX' = -RX \tag{5.8}$$

This transformation expands the reachable orientation set and guarantees that a pickup configuration is always obtained. Since this orientation may not strictly satisfy admissibility constraints, it is marked as *admissibility not confirmed*. This status is propagated to downstream modules, allowing conservative execution strategies to be applied.

Algorithm 2 High-level orientation selection strategy.

```
Retrieve tracked sheet pose and active zone
Load zone-specific admissibility window {R_Z^min, R_Z^max}
Validate pickup orientation and apply correction if required
Compute reference-face equivalent orientation
if reference-face orientation is admissible then
    Select reference-face orientation
else if detected orientation is admissible then
    Select detected orientation
else
    Compute inverted orientation
    Select inverted orientation (admissibility not confirmed)
end if
Forward orientation to downstream planning modules
```

Algorithm 2 summarises the implemented selection strategy at an abstract level without exposing implementation-specific details.

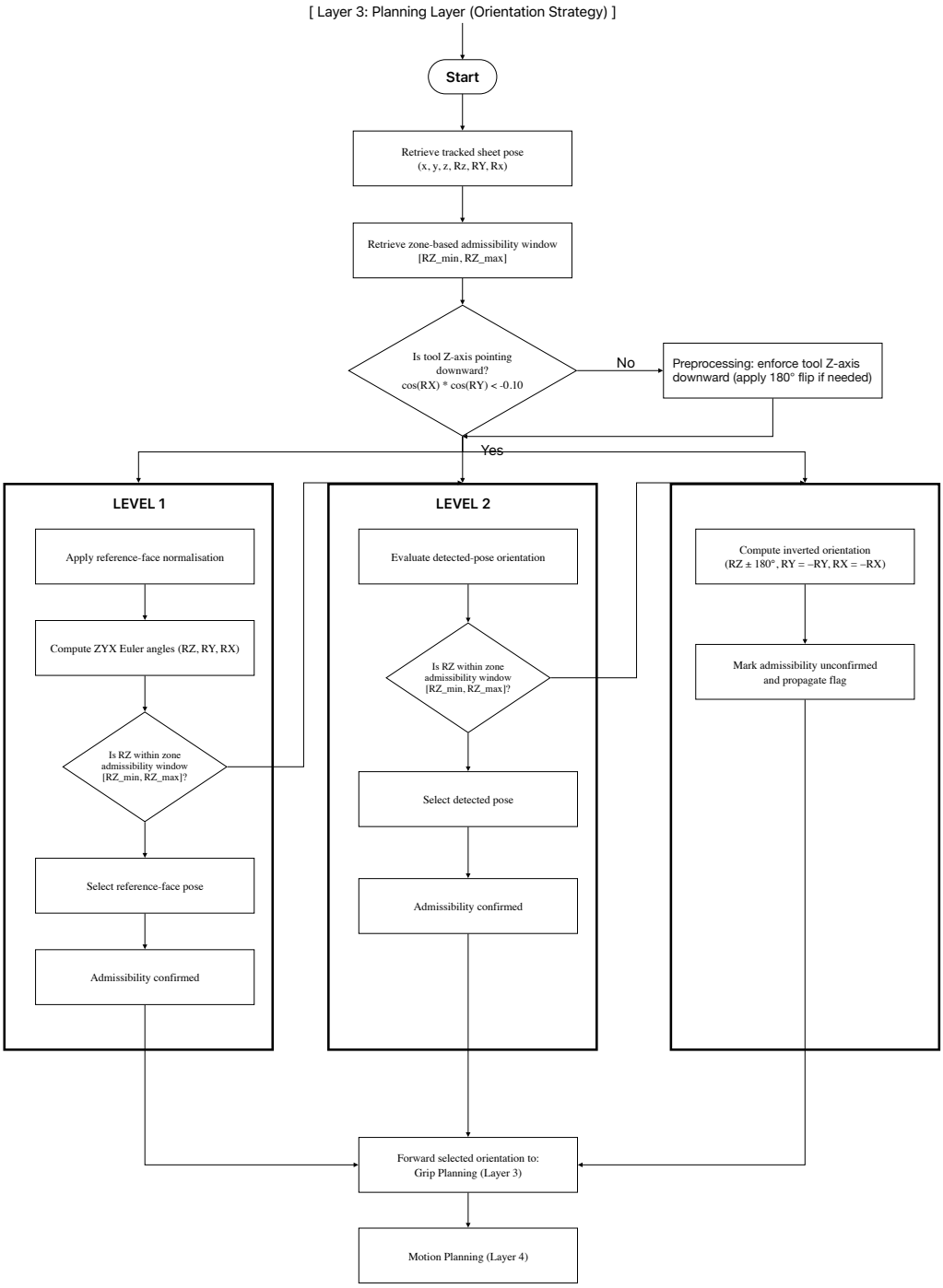


Figure 5.10: Three-level orientation selection strategy for determining a valid pickup pose under kinematic constraints. *Source: own work.*

Strategy Outcome The proposed strategy guarantees that a valid pickup orientation is always determined while prioritising kinematic feasibility and geometric consistency. By integrating admissibility constraints with a structured fallback mechanism, the method directly supports pre-execution validation and reduces the likelihood of infeasible motion generation. The selected orientation is then passed to subsequent modules for grip planning and trajectory generation.

5.6 Workpiece Handling and Gripper Control

Once a valid pickup orientation has been determined, the system must establish a reliable physical interaction between the gripper and the sheet while maintaining consistency between the physical robot state and the virtual simulation. Any mismatch between the vacuum state of the gripper and the attachment state in the simulation leads to incorrect pose propagation and invalid trajectory computation in subsequent stages. This consistency is ensured through two coordinated mechanisms: digital IO control of the vacuum gripper and kinematic object attachment within the simulation environment.

5.6.1 Vacuum Gripper Digital Input and Output Sequence

The vacuum gripper is actuated using two digital output signals mapped within the drag&bot IO management system. One signal activates the vacuum by closing the suction valve, while the other releases the vacuum by opening the exhaust valve. These signals are strictly mutually exclusive; simultaneous activation is avoided to prevent conflicting pressure states and potential hardware damage.
Control commands are issued via the `/robot/set_digital_ios` interface, which transmits structured IO instructions specifying the target output and its Boolean state. To ensure reliable gripping, a dwell period of approximately 250 ms is applied after vacuum activation. This delay allows the suction cups to establish a stable seal with the sheet surface, which is necessary for maintaining grasp stability during subsequent motion [28].
A secondary delay of approximately 150 ms is introduced after each IO operation to ensure that the state transition propagates through the communication layer before further commands are issued. This prevents timing inconsistencies between command execution and runtime state updates.

5.6.2 Kinematic Object Attachment and Detachment

Within the simulation environment, the physical coupling between the gripper and the sheet is represented through kinematic attachment. When the gripper successfully establishes contact, the corresponding object is attached to the robot tool frame, ensuring that subsequent robot motions are reflected in the object pose.

When the vacuum is released, the object is detached and remains fixed in its last known position.
Attachment is performed using the `/dnb_object_manager/attach` interface without enforcing a position override. This avoids introducing discontinuities between the tracked object pose and the simulated state, as the tracked pose may not exactly coincide with the instantaneous tool frame position. Position override is reserved only for controlled initialization procedures where explicit alignment is required.
Detachment is executed using the `/dnb_object_manager/detach` interface within the same reference frame. To maintain consistency, attachment and detachment operations are always executed in a fixed sequence together with the corresponding IO commands:

- Grasp sequence: vacuum ON $\rightarrow$ dwell $\rightarrow$ attach
- Release sequence: vacuum OFF $\rightarrow$ dwell $\rightarrow$ detach

This sequence is encapsulated within centralized helper routines that are invoked by all modules performing grasp or release operations. Centralizing this logic ensures that the physical and virtual system states remain synchronised at all times and prevents inconsistent conditions in which the vacuum state and attachment state do not correspond.
With physical grasping and state synchronisation established, the next step is to determine feasible suction cup positions on the workpiece surface.

5.7 Grip Position Planning and Feasibility Validation

Following the determination of a valid pickup orientation and the establishment of a consistent gripper control mechanism, the system must identify feasible suction cup placements on the sheet surface. Arbitrary placement is not sufficient, as it may violate geometric constraints or fail to provide adequate holding force during handling. The grip planner therefore evaluates both geometric feasibility and physical stability to ensure a reliable and executable grasp configuration. This structured approach directly addresses the limitations identified in Chapter 2, where manual or heuristic-based gripping strategies lack consistency and adaptability. This approach enables deterministic and repeatable grip planning, which is essential for automated program generation in an industrial environment.

5.7.1 Sheet Geometry and Hole Detection

The sheet is modelled as a planar rectangular surface with uniform mass distribution, defined by its length, width, thickness, and mass obtained from the configuration. If the mass is not explicitly specified, it is estimated from the sheet volume and the material density defined in the configuration.

In addition to the outer boundary, internal features such as holes are extracted from the STL mesh during the geometry analysis stage. Each hole is represented by its centre position and radius in the sheet coordinate frame. These regions are explicitly excluded from the feasible gripping area, since suction cups cannot form a stable seal over discontinuities. This ensures that only physically valid contact regions are considered during grip planning.

5.7.2 Vacuum Holding Force Validation

For each candidate grip configuration, the system evaluates whether the available vacuum force is sufficient to retain the sheet under both gravitational and dynamic loading conditions. The contact area of a single suction cup is defined as

$$A_{\text{cup}} = \pi \left(\frac{d_{\text{cup}}}{2} \right)^2 \tag{5.9}$$

The total available holding force generated by n suction cups is expressed as

$$F_{\text{vacuum}} = n \cdot \Delta P \cdot A_{\text{cup}} \cdot \eta \tag{5.10}$$

where ΔP represents the pressure differential and η accounts for efficiency losses due to leakage and surface conditions.
The required holding force is determined as

$$F_{\text{required}} = SF \cdot m \cdot \sqrt{a_{xy}^2 + (g + a_z)^2} \tag{5.11}$$

where SF is a safety factor and a_{xy}, a_z represent dynamic accelerations.
A candidate configuration is considered feasible only if

$$F_{\text{vacuum}} \geq F_{\text{required}} \tag{5.12}$$

This formulation follows standard principles of vacuum-based gripping and ensures that only configurations capable of maintaining stable contact under motion are considered for execution [29].

5.7.3 Grip Candidate Search and Selection

Candidate suction cup positions are generated using a multi-resolution grid-based search over the sheet surface. The process begins with a coarse discretisation to identify feasible regions, followed by progressive refinement to improve placement accuracy in promising areas. Additional candidates are generated near hole boundaries to maximise usable surface area while maintaining required clearances.
Each candidate position is evaluated against geometric constraints. The cup centre must satisfy minimum edge clearance requirements, avoid overlap with hole regions including a defined buffer zone, and comply with spacing constraints in multi-cup configurations. For the two-cup case, candidate positions are evaluated in pairs,

with the inter-cup distance constrained to ensure sufficient grip span across the workpiece.
After geometric filtering, candidates are further evaluated using the vacuum holding force criterion described in Section 5.7.2. Only configurations satisfying both geometric and physical constraints are retained.
The remaining candidates are ranked using a scoring function that balances multiple criteria, including edge clearance, distance from internal features, and spatial distribution across the sheet. In multi-cup configurations, the scoring additionally promotes wider spacing between cups to improve stability during transport. This structured evaluation replaces heuristic placement strategies commonly found in manual programming approaches and ensures consistent selection of robust grip configurations.
The implemented search and selection procedure is summarised in Algorithm 3.
Algorithm 3 describes the grip candidate search and selection procedure.

Algorithm 3 Grip candidate search and selection procedure

```
Load sheet geometry and grip-planning parameters
Generate candidate grip positions on the sheet surface using grid-based search
Exclude candidates violating edge-clearance constraints
Exclude candidates overlapping with hole regions and buffer zones
if no feasible candidates remain then
    Terminate grip planning
end if
if two-cup configuration is required then
    Form candidate pairs
    Exclude pairs violating minimum spacing constraints
end if
Validate feasible configuration(s) using the vacuum holding force criterion
Score remaining configuration(s) based on geometric suitability
Select the highest-scoring grip configuration
Compute approach, attachment, and retreat waypoints (A1, B1, R1)
Forward selected grip result to trajectory generation
```

The highest-scoring configuration is selected as the final grip position. Based on this selection, three key waypoints are derived: the approach pose (A1), the attachment pose (B1), and the retreat pose (R1). These poses are computed relative to the sheet surface and tool geometry, ensuring a consistent and collision-free pickup sequence.
Figure 5.11 illustrates the complete grip planning pipeline, including force validation, candidate generation, constraint filtering, scoring, and waypoint derivation.
The grip candidate search and selection process described above can be illustrated through a simplified implementation that captures the essential steps of the planner. The approach involves systematic candidate generation over the sheet surface

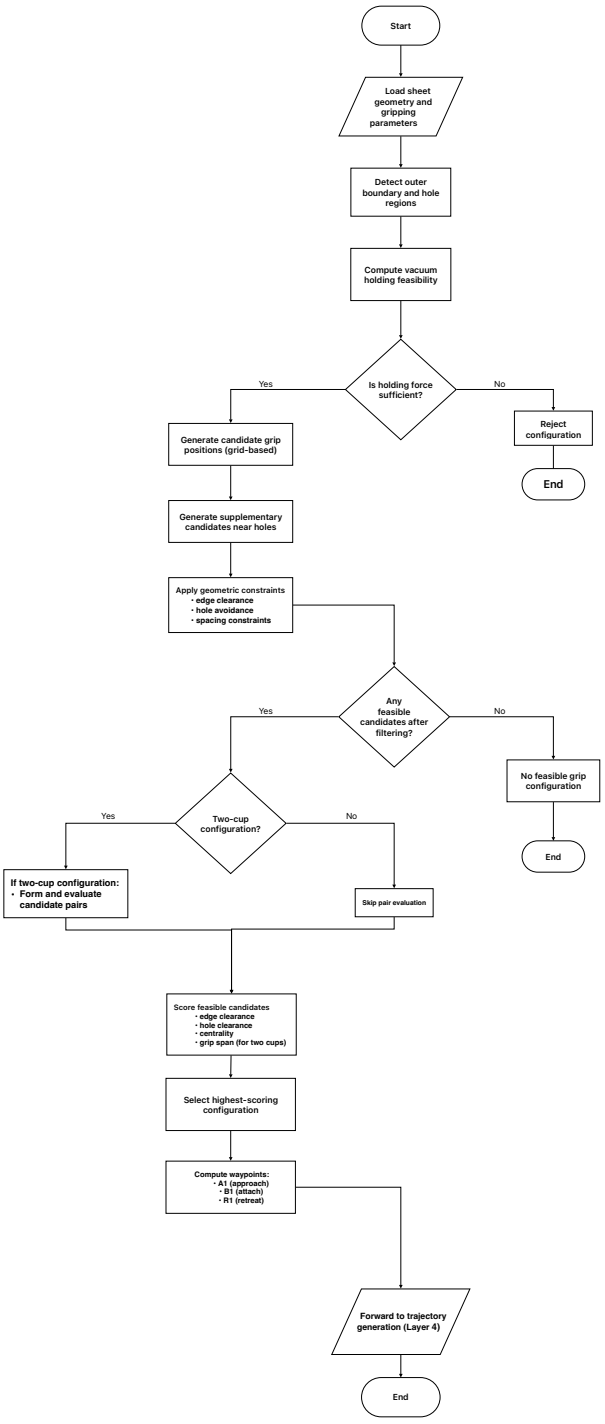


Figure 5.11: Grip position planning and feasibility validation workflow. The process integrates geometric filtering and vacuum force validation to ensure physically stable and executable grasp configurations prior to trajectory generation. *Source: own work.*

using a grid-based discretisation, followed by geometric filtering to remove invalid positions, and finally a scoring-based selection of the most suitable grip point. The simplified implementation shown below highlights the core logic of this process without exposing implementation-specific details.
Listing 5.3 presents a simplified Python implementation of grid-based grip candidate generation, geometric constraint filtering, and final grip position selection.

```python
import math

def generate_candidates(length_m, width_m, step_m):
    return [
        (x, y)
        for x in frange(-length_m/2, length_m/2, step_m)
        for y in frange(-width_m/2, width_m/2, step_m)
    ]


def is_valid(point, length_m, width_m, min_edge_clear_m, holes,
    hole_buffer_m):
    x, y = point

    # Edge clearance check
    if abs(x) > length_m/2 - min_edge_clear_m:
        return False
    if abs(y) > width_m/2 - min_edge_clear_m:
        return False

    # Hole avoidance check
    for h in holes:
        if math.hypot(x - h["x"], y - h["y"]) < h["radius"] +
            hole_buffer_m:
             return False

    return True


def select_best(candidates):
    return max(candidates, key=lambda p: -(abs(p[0]) + abs(p[1])))
```

Listing 5.3: Simplified Python implementation of grid-based grip candidate generation, geometric constraint filtering, and final grip position selection.

5.8 Regrip Planning and Execution

Even when a valid pickup orientation and a feasible grip configuration are established, the current grasp may not permit the robot to reach the required bending pose. This limitation arises when the target configuration lies outside the reach-

able workspace or when kinematic constraints, such as joint limits or proximity to singularities, prevent a valid motion solution [27]. To address this, the system implements an automatic regrip planning strategy that determines the required reorientation dynamically. This approach directly addresses the limitations identified in the state of the art, where regrip operations are typically predefined or manually programmed.

5.8.1 Regrip Decision Strategy

In contrast to conventional approaches based on fixed regrip sequences, the proposed system evaluates a discrete set of candidate workpiece orientations and selects a configuration that satisfies kinematic feasibility. This enables the system to determine both the required orientation and the number of regrip steps automatically.
The candidate set consists of discrete orientation increments corresponding to physically achievable rotations on the orientation table. For each candidate orientation, the system performs the following sequence:

- Update the workpiece pose according to the candidate orientation
- Determine a valid pickup orientation using the strategy described in Section 5.5
- Compute feasible grip positions using the planner described in Section 5.7
- Evaluate kinematic feasibility using inverse kinematics and trajectory validation

A candidate orientation is accepted only when all feasibility conditions are satisfied. The evaluation follows a sequential selection strategy in which candidates are tested in a predefined order, and the first feasible solution is selected. This ensures computational efficiency while maintaining robustness for real-time execution.
The number of required regrip steps is implicitly determined from the difference between the current orientation and the selected feasible configuration. This approach avoids unnecessary intermediate operations while ensuring that the final pose remains reachable and executable within the robot workspace.
Algorithm 4 summarises the decision process. The evaluation reuses the existing planning pipeline, ensuring consistency with orientation selection, grip planning, and trajectory generation.

5.8.2 Regrip Execution Sequence

Once a feasible orientation has been identified, the system executes a structured regrip sequence to realise the required reorientation. The sequence consists of the following steps:

Algorithm 4 Regrip orientation selection.

```
Define candidate orientation set
for each candidate orientation do
    Compute updated workpiece pose
    Evaluate pickup orientation
    Compute grip configuration
    Check kinematic feasibility
    if feasible then
        Select orientation
        break
    end if
end for
Determine required number of regrip steps
```

1. Transport the workpiece from the bending station approach position to the orientation table deposit position while maintaining the current grasp
2. Release the workpiece through controlled vacuum deactivation and kinematic detachment
3. Move the robot to a retract standoff position to ensure collision-free clearance
4. Re-approach the workpiece at the updated orientation and perform the grasp sequence
5. Repeat the regrip cycle if additional orientation adjustments are required
6. Perform the final grasp and return the workpiece to the bending station

Each motion segment is executed using smooth trajectories to minimise dynamic loads on the gripper and ensure stable handling. Intermediate standoff positions provide safe transitions between grasping and releasing operations.

5.8.3 System Integration and Configuration

The regrip execution sequence operates within the existing motion framework and uses configuration-defined target positions, including deposit, re-grasp, retract, and approach poses. These parameters define the physical layout of the cell but do not determine the number of regrip steps.

The decision to trigger a regrip is handled by the planning orchestrator, which evaluates feasibility based on orientation admissibility and kinematic constraints. This separation between decision-making and execution ensures modularity and allows the regrip strategy to be extended without modifying the motion execution layer.

Compared to configuration-driven approaches, the proposed method improves adaptability by determining the minimum required number of regrip steps dynamically. At the same time, it maintains computational efficiency by evaluating a

structured set of candidate orientations rather than performing global optimisation. This balance makes the approach suitable for industrial deployment, where predictable and robust behaviour is required.

5.9 Opposite-Face Bending Workflow

After completing all primary-side bending operations, the system evaluates whether additional bends are required on the opposite face of the sheet. These operations cannot be executed directly from the initial pickup configuration, as the required face is not accessible to the press brake tool. Consequently, the workpiece must be reoriented through a controlled flip operation before continuing the bending process.

5.9.1 Pre-Flip Verification and Transition Correction

Before initiating the flip sequence, the system verifies that the current sheet pose corresponds to the expected reference configuration. This verification is performed using a wrap-safe six-degree-of-freedom pose comparison, with a positional tolerance of 2 mm and an angular tolerance of 1°.

This validation step is required because the sheet may have undergone regrip operations during the primary bending sequence. Any deviation from the expected pose would propagate errors into subsequent motion planning stages, leading to incorrect trajectory generation.

If the verification fails, a transition correction step is executed. The system first restores the reference orientation using the strategy described in Section 5.5. A controlled pickup and repositioning sequence is then performed to re-establish a consistent and verified alignment between the robot and the workpiece.

If the verification is successful, the correction step is omitted and the system proceeds directly to the flip sequence.

5.9.2 Sheet Flip Sequence and Opposite-Face Bending Execution

The flip operation is performed using a dedicated station within the robotic cell. The workpiece is transferred through two predefined reference positions to achieve a physically feasible reorientation.

First, the robot transports the sheet to the flip station placement position while maintaining the current grasp. The workpiece is then released in a controlled manner. The robot subsequently moves to an intermediate standoff pose and re-approaches the sheet from the opposite side, establishing a new grasp that results in the flipped orientation.

This two-stage transfer is preferred over a direct in-gripper rotation. A direct rotation would require a continuous motion that exceeds the kinematic capabilities of the robot and may violate workspace or collision constraints [27].

Following the flip operation, the sheet is aligned such that the previously inaccessible face is oriented towards the press brake tool. The system then applies the same validation pipeline used for primary-side bending, including zone classification, orientation determination, grip position planning, and trajectory generation.
Reusing the same processing pipeline ensures consistent system behaviour across both primary and opposite-side operations. It also avoids the need for separate handling logic, thereby maintaining modularity and reducing system complexity.
Figure 5.12 illustrates the complete opposite-side bending workflow, including pre-flip verification, optional transition correction, flip execution, and subsequent bending operations.
After completing all opposite-side bends, any remaining staged operations are executed using the existing handling framework. The system then proceeds to the final unloading sequence.
ints, and executed by the robot as a continuous motion from the start to the target pose.

5.10 Trajectory Generation

Trajectory generation constitutes the final stage of the planning pipeline, where validated task-level poses are transformed into executable robot motion. The input to this stage is an ordered sequence of TCP poses obtained from upstream modules, including grip planning, regrip handling, and opposite-face operations. Each pose defines the desired end-effector position and orientation within the robot world frame.
Rather than executing direct point-to-point transitions, the system constructs a continuous and smooth trajectory composed of intermediate waypoints. This approach ensures kinematic feasibility, reduces dynamic loading, and improves execution stability during sheet handling.

5.10.1 Three-Phase Linear Trajectory Structure

Each motion segment is generated using a structured three-phase trajectory: lift, lateral transfer, and descent.
In the lift phase, the TCP is translated vertically to a predefined safe height to avoid collisions with the workpiece, fixtures, or surrounding environment. In the lateral transfer phase, the robot moves towards the target position while maintaining this clearance height. During this phase, the end-effector orientation is constrained to limit excessive joint rotations and reduce proximity to kinematic singularities. Finally, in the descent phase, the TCP is lowered to the target pose, where the required orientation for task execution is fully applied.
To further improve motion feasibility, an intermediate rotation pose is introduced above the target location. This ensures that major orientation adjustments are completed before the final approach, reducing the risk of instability near contact

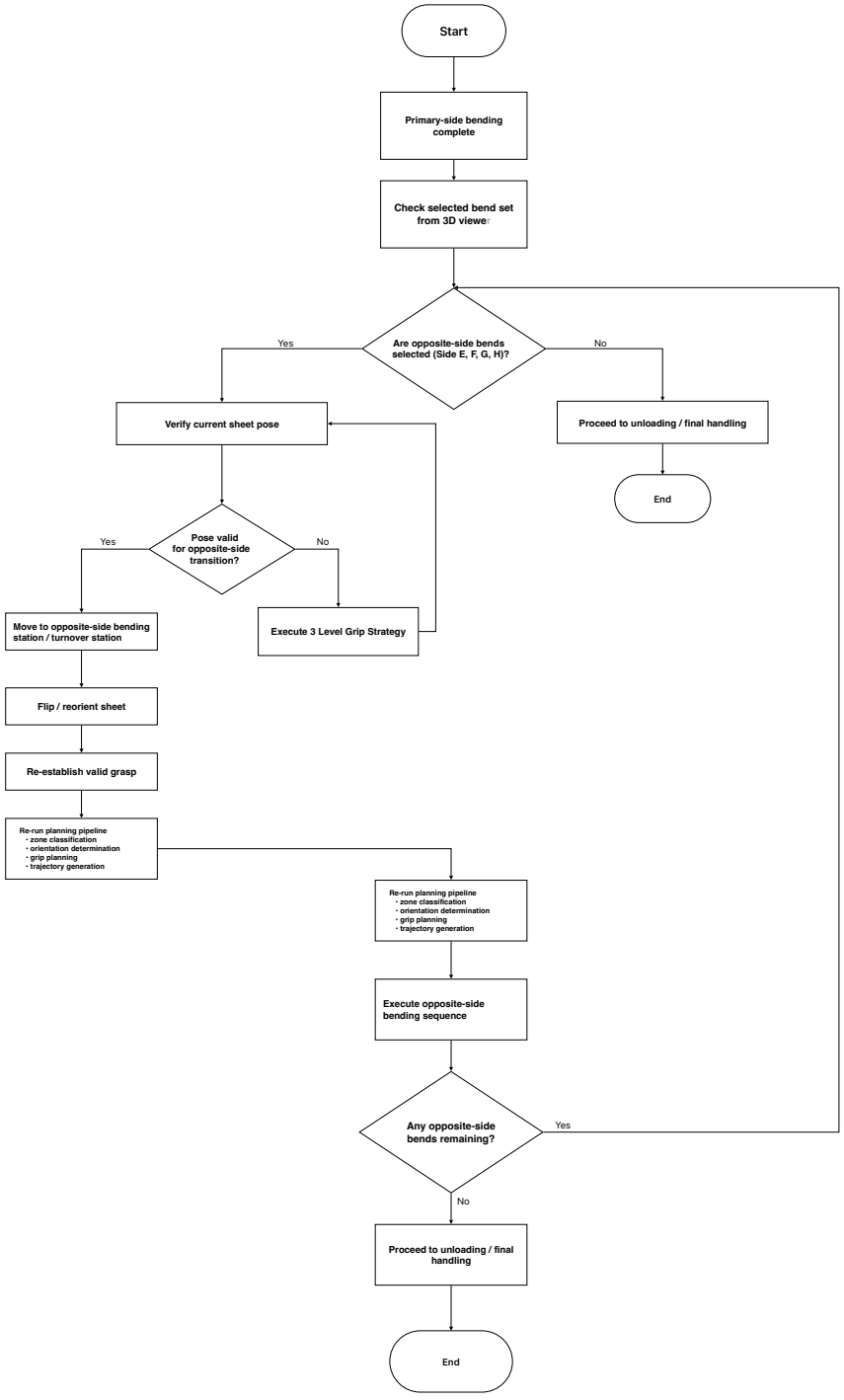


Figure 5.12: Opposite-face flip workflow enabling access to previously unreachable bending surfaces. *Source: own work.*

regions.

5.10.2 Waypoint Generation and Interpolation

The trajectory between successive poses is generated through interpolation in Cartesian space. For a start pose $\mathbf{T}_s$ and a target pose $\mathbf{T}_t$, the translational component is computed as

$$\mathbf{p}(\lambda) = (1 - \lambda)\mathbf{p}_s + \lambda\mathbf{p}_t, \tag{5.13}$$

where $\lambda \in [0, 1]$ is the interpolation parameter.
Rotational interpolation is performed using spherical linear interpolation (SLERP) in quaternion space, which provides smooth and constant-speed rotation between orientations [30].
To ensure smooth velocity and acceleration profiles, the interpolation parameter follows a minimum-jerk trajectory given by

$$s(u) = 10u^3 - 15u^4 + 6u^5, \tag{5.14}$$

which guarantees zero velocity and acceleration at the start and end of each motion segment [31]. Such profiles are widely adopted in robotic trajectory planning to achieve smooth and physically consistent motion behaviour [32].
By sampling this progression at discrete intervals, a sequence of intermediate TCP waypoints is generated that approximates the continuous trajectory.

```
def min_jerk(u):
    return 10*u**3 - 15*u**4 + 6*u**5

waypoints = []
for k in range(N):
    u = k / (N - 1)
    s = min_jerk(u)
    p = (1 - s) * p_start + s * p_target
    r = slerp(r_start, r_target, s)
    waypoints.append((p, r))
```

Listing 5.4: Illustrative waypoint generation using minimum-jerk interpolation

The number of waypoints is determined based on the path length and a predefined resolution parameter, ensuring a balance between trajectory smoothness and computational efficiency.

```
distance = euclidean_dist(p_start, p_target)
N = max(2, math.ceil(distance / delta_s_max))
samples = [k / (N - 1) for k in range(N)]
```

Listing 5.5: Illustrative determination of waypoint count based on path length

5.10.3 Kinematic Mapping and Joint Execution

Each interpolated TCP waypoint is mapped to a corresponding robot joint configuration using inverse kinematics. For each waypoint, a feasible solution is selected based on continuity with the previous configuration, avoiding abrupt joint changes and ensuring smooth motion across the trajectory.
As the robot executes the sequence, all joints evolve continuously, resulting in a smooth end-effector motion that closely follows the planned Cartesian path.

5.10.4 Workspace Constraints and Singularity Handling

To ensure execution feasibility, each waypoint is validated against workspace constraints. The TCP pose is transformed into the corresponding flange pose, and its position is checked against the allowable workspace limits. If a waypoint exceeds these limits, it is projected onto the nearest feasible boundary.
Singularity avoidance is addressed by restricting orientation changes during the lateral motion phase. Large orientation adjustments are deferred to the approach phase, where the robot operates in a more stable configuration. This reduces the risk of excessive joint velocities and improves overall motion robustness.

5.10.5 Motion Execution and Path Reduction

The interpolation process generates a dense set of waypoints, which may exceed the processing capacity of the robot controller. To address this, a path reduction strategy is applied, retaining only those waypoints that exceed predefined positional or rotational thresholds. This preserves the essential characteristics of the trajectory while reducing communication overhead.
The resulting waypoint sequence is transmitted as a series of linear motion commands, which are executed by the robot controller using blending techniques. This allows smooth transitions between consecutive waypoints without stopping, producing continuous and stable motion.
Overall, the trajectory generation module forms a deterministic transformation pipeline in which validated task-level poses are converted into executable robot motion through interpolation, kinematic mapping, constraint validation, and controlled execution.

5.11 System Reliability and Failure Mode Analysis

To evaluate the robustness and operational reliability of the implemented system, a Failure Mode and Effects Analysis (FMEA) is conducted. This structured methodology identifies potential failure modes within the robotic bending workflow, evaluates their impact on system performance, and assesses the effectiveness of implemented mitigation strategies.

FMEA is based on three evaluation criteria: severity (S), occurrence (O), and detectability (D). These factors are combined to compute the Risk Priority Number (RPN), defined as

$$\text{RPN} = S \times O \times D \tag{5.15}$$

Each parameter is assessed using a qualitative scale from 1 (low) to 10 (high), where higher RPN values indicate higher-priority risks requiring mitigation.

5.11.1 Identification of Failure Modes

The analysis focuses on critical stages of the execution pipeline, including pickup orientation, grip planning, trajectory execution, regrip handling, and opposite-face operations. The identified failure modes are summarised in Table 5.2.

Failure Mode	Effect	S	O	D	RPN	Cause	Mitigation
Sheet unreachable during pickup	Robot stops mid-trajectory; session aborted	7	3	3	63	Invalid spatial placement or orientation outside admissible zone	Spatial zone classification and orientation constraints applied before motion; system halts with operator notification
Incorrect grip position (cup over hole)	Vacuum failure; sheet may fall during transport	8	2	2	32	Incomplete feature detection in hole map	Clearance offsets applied around detected holes; final validation ensures no suction cup overlaps with holes
Vacuum holding force insufficient	Sheet slips or drops during motion	8	2	1	16	Insufficient gripping force due to load or acceleration	Force feasibility check applied for each candidate grip; safety factor enforced and invalid grips rejected
Motion command rejected by controller	Robot stops; process interruption	6	3	2	36	Invalid or inconsistent motion command format	Command validation and structured message formatting ensure consistent communication with controller
Trajectory exits workspace boundary	Motion infeasible; execution halted	6	4	3	72	Target pose near or beyond reachable workspace limits	Workspace clamping applied to all trajectory points; limits enforced on radial and vertical reach
Inverse kinematics failure due to excessive attachments	Robot cannot compute valid configuration	5	2	3	30	Multiple simultaneous object attachments causing solver overload	Strict attach-detach discipline enforced; each attachment preceded by release of previous objects
Self-collision in simulation	Invalid motion path generated without warning	4	3	5	60	Lack of self-collision detection in simulation environment	Three-phase trajectory structure reduces collision risk; limitation acknowledged for physical deployment
Configuration error (invalid or missing parameters)	System fails at startup; no execution	3	3	1	9	Missing or incorrectly defined configuration parameters	Configuration validation performed at startup; errors detected before motion execution

Table 5.2: Failure Mode and Effects Analysis (FMEA) of the robotic bending system.

The selected failure modes reflect the most critical dependencies in the system, particularly those related to geometric accuracy, kinematic feasibility, and physical interaction between the robot and the workpiece.

5.11.2 Mitigation Strategies

Several design measures have been incorporated to reduce both the likelihood and impact of identified failure modes.
Geometric consistency is ensured through structured validation stages, including fixture pose deviation checks and orientation admissibility constraints. These mechanisms prevent incorrect pose assumptions from propagating into motion generation.
Grip reliability is addressed through combined geometric filtering and physical feasibility validation. By enforcing edge clearance, hole avoidance, and vacuum force constraints, the system ensures that only stable grip configurations are selected.
Trajectory-related risks are mitigated through interpolation-based motion generation and workspace validation. The use of minimum-jerk profiles and waypoint filtering ensures smooth motion while avoiding excessive controller load. In addition, singularity avoidance strategies and orientation constraints are applied to maintain kinematic stability throughout execution.
Regrip and flip operations are implemented as structured sequences with defined intermediate poses, ensuring collision-free transitions and consistent object handling. The use of deterministic execution pipelines further reduces uncertainty by enforcing a fixed order of validated operations.

5.11.3 Discussion

The FMEA results indicate that the most critical risks are associated with pose accuracy and trajectory feasibility, as these directly influence all downstream planning and execution stages. These risks are effectively mitigated through layered validation, constraint-based planning, and structured motion generation.
Overall, the system demonstrates a balanced reliability profile in which high-severity failure modes are either low in occurrence or mitigated through strong detection and validation mechanisms. This confirms that the proposed architecture not only enables automated bending operations but also ensures safe and predictable execution under varying operating conditions.

5.11.4 Conclusion

This chapter presented the complete implementation of the proposed system for automated press brake sheet metal bending, based on the selected Concept B. The implementation translates the conceptual design into a fully functional software architecture that integrates geometric reasoning, constraint-based planning, and robotic motion execution within a unified framework.

A layered system architecture was developed to ensure modularity, testability, and strict enforcement of execution order. Each functional component, including orientation determination, grip planning, regrip handling, and trajectory generation, was designed as an independent module with clearly defined responsibilities. This structure enables traceability between system requirements and implementation, while also supporting future extensibility.

The system demonstrates the capability to generate robot programs automatically from configuration data and geometric inputs, eliminating the need for manual waypoint definition. By incorporating structured validation mechanisms at multiple stages, including pre-execution safety checks, grip feasibility validation, and workspace-constrained trajectory generation, the system ensures reliable and safe operation.

A key contribution of this work lies in the integration of adaptive regrip planning and opposite-face handling within a deterministic execution pipeline. Unlike conventional approaches that rely on predefined sequences, the system dynamically evaluates feasible configurations and determines the required manipulation strategy at runtime. This improves flexibility while maintaining computational efficiency suitable for industrial applications.

The trajectory generation framework further enhances system robustness by combining smooth interpolation techniques with kinematic validation and path reduction strategies. This ensures that generated motions are both executable by the robot controller and stable under real-world operating conditions.

The conducted Failure Mode and Effects Analysis confirms that the system effectively mitigates critical risks through layered validation and structured execution. High-risk scenarios related to pose accuracy, gripping stability, and trajectory feasibility are addressed through a combination of detection mechanisms and constraint-based design.

Overall, the implemented system provides a robust and scalable solution for automated sheet metal bending. It achieves a balance between flexibility, safety, and computational efficiency, demonstrating the feasibility of fully automated robot program generation in an industrially relevant context. This implementation forms a strong foundation for further evaluation and validation, as presented in the subsequent chapter.

5.12 Requirement Compliance Matrix

To ensure full traceability between the system requirements defined in Chapter 3 and the implemented solution, a requirement compliance matrix is presented in Table 5.3. The matrix establishes a direct mapping between each requirement and its corresponding implementation component within the system.

Both functional requirements (FR) and non-functional requirements (NFR) are included. Each requirement is linked to the relevant module, algorithm, or validation

mechanism described in this chapter, ensuring that all specified requirements are systematically addressed within the implemented system.

Requirement ID	Requirement Description	Implementation Mapping
FR-00	Overall system shall automate press brake bending process workflow	Achieved through the integrated planning pipeline, including orientation strategy (Section 5.5), grip planning (Section 5.7), regrip planning (Section 5.8), and trajectory generation (Section 5.10)
FR-01	Automatic generation of robot programs from geometric input	Implemented through the structured planning pipeline combining orientation determination, grip planning, and trajectory generation (Sections 5.5, 5.7, and 5.10)
FR-02	Support for multiple workpiece orientations and configurations	Handled through the three-level orientation strategy with admissibility validation and fallback mechanism (Section 5.5)
FR-03	Automatic grip position determination	Achieved through constraint-based grip planning with geometric filtering and force validation (Section 5.7)
FR-04	Handling of regrip scenarios for unreachable configurations	Implemented via dynamic regrip planning and execution strategy based on kinematic feasibility evaluation (Section 5.8)
FR-05	Support for opposite-side bending operations	Enabled through the flip workflow and reuse of the planning pipeline for secondary operations (Section 5.9)
FR-06	Seamless communication with robot runtime	Achieved using the AnyRobot Interface with authenticated WebSocket communication (Section 5.2.3)
FR-07	Pre-execution safety validation before motion execution	Implemented through fixture pose validation, operator confirmation, and runtime monitoring mechanisms (Section 5.4)
NFR-01	Minimal operator interaction during normal execution	Achieved through bend-side selection, execution confirmation, and integrated operator interaction in the drag&bot cockpit (Section 5.2.5)
NFR-02	Configurability without source code modification	Achieved through centralized YAML configuration architecture and parameter-driven system design (Section 5.2.4)
NFR-03	Clear warning and error messages for unsafe or infeasible conditions	Implemented through warning generation, modal operator notifications, and controlled stop handling (Section 5.4)
NFR-04	Robust behaviour under repeated execution cycles	Ensured through a structured modular pipeline and fixed execution order across system layers (Sections 5.2–5.12)
NFR-05	Pose deviation tolerance within defined limits	Implemented through configurable pose-deviation thresholds, fixture deviation checks, and controlled-stop handling (Section 5.4)

Table 5.3: Requirement compliance matrix linking system requirements to implementation components.

The compliance matrix demonstrates that all functional and non-functional requirements defined in Chapter 3 are fully addressed within the implemented system. Each requirement is mapped to a specific architectural component or validation mechanism, ensuring complete traceability between system design and implementation. This structured mapping confirms that the developed solution consistently satisfies the objectives established in earlier chapters while maintaining coherence across the entire system pipeline.

6 Experimental Evaluation and Validation

6.1 Experimental Setup

To evaluate the performance and robustness of the implemented system, a structured experimental setup is defined. The evaluation is conducted under controlled conditions to ensure repeatability and consistency across all test scenarios. This section outlines the test environment, the selected scenarios, and the metrics used to assess system performance.

6.1.1 Test Environment

All experiments are conducted within the virtual environment used during system development. This ensures consistency between implementation and evaluation, allowing the behaviour of the system to be assessed under identical conditions.
The evaluation setup includes the complete system pipeline operating on simulated workpiece data. The environment enables controlled testing of different scenarios without external disturbances, ensuring that observed results are attributable to the system behaviour rather than environmental variability. This approach supports repeatable experimentation and reliable comparison across multiple test runs.

6.1.2 Test Scenarios

To assess the adaptability of the system, a set of representative test scenarios is defined based on variations in workpiece geometry and initial placement conditions. These scenarios are designed to reflect the key operational challenges addressed by the system.
The evaluated scenarios include:

- Variations in sheet dimensions and mass properties
- Arbitrary initial orientations of the workpiece
- Placement across different spatial regions of the workspace
- Configurations requiring regrip operations due to kinematic constraints
- Tasks involving both primary and opposite-face bending operations

Each scenario is executed multiple times to verify that the system produces consistent and repeatable results under identical conditions.

6.1.3 Evaluation Metrics

The system performance is assessed using a combination of quantitative and qualitative metrics. These metrics are selected to evaluate both functional correctness and operational efficiency.
The primary evaluation metrics include:

- **Execution success rate:** proportion of scenarios completed without failure
- **Planning time:** time required to generate a complete motion plan
- **Execution time:** total duration of the bending operation
- **Number of regrip operations:** indicator of handling efficiency
- **Failure occurrence:** identification and classification of unsuccessful executions

In addition, qualitative observations are used to assess motion smoothness, stability of gripping, and overall system behaviour during execution.
Together, the defined environment, scenarios, and metrics provide a consistent and comprehensive basis for evaluating the effectiveness of the implemented system.

6.2 Functional Validation

This section evaluates the correctness of the core functional components of the implemented system. Each module of the planning pipeline is assessed based on its ability to produce valid and consistent outputs under varying input conditions. The validation focuses on confirming that the system behaves as expected rather than describing how each module operates.

6.2.1 Orientation Strategy Validation

The orientation strategy is evaluated using scenarios in which the workpiece is placed in arbitrary initial poses across different spatial regions. The results confirm that the system consistently determines a feasible pickup orientation for all tested configurations.
In cases where the initial orientation satisfies admissibility constraints, the detected pose is preserved. When this is not possible, the system successfully applies corrective transformations or fallback strategies to generate a valid orientation. Across all evaluated scenarios, a feasible pickup orientation is obtained without requiring manual adjustment, demonstrating robust handling of non-deterministic input conditions.

6.2.2 Grip Planning Validation

The grip planning module is validated by assessing its ability to generate stable and feasible suction cup placements under geometric and physical constraints. Test cases include variations in workpiece size and configurations containing internal cut-outs.
The results show that all selected grip configurations satisfy geometric constraints, including edge clearance and exclusion of hole regions. Furthermore, the force validation step ensures that only configurations with sufficient holding force are accepted. In multi-cup scenarios, the system consistently selects configurations that improve stability through appropriate spacing. These results confirm that the grip planner produces reliable and physically feasible solutions.

6.2.3 Regrip Planning Validation

The regrip planning strategy is evaluated using scenarios in which the initial configuration is not directly executable due to kinematic limitations. The results indicate that the system correctly identifies such cases and determines an alternative configuration that enables successful execution.
For all evaluated scenarios requiring regrip, the system selects a feasible orientation and completes the required reorientation sequence without failure. The number of regrip operations is minimized through the selection strategy, confirming that the approach avoids unnecessary intermediate steps while maintaining feasibility.

6.2.4 Opposite-Face Bending Validation

The opposite-face bending workflow is validated through scenarios that require bending operations on both sides of the workpiece. The evaluation confirms that the system successfully performs the flip operation and continues execution using the same planning pipeline.
After the flip, the system re-establishes a valid configuration and completes the remaining bending operations without inconsistencies. The results demonstrate that multi-stage operations are handled seamlessly, without requiring separate control logic for different workpiece orientations.
Overall, the functional validation confirms that all major components of the system operate correctly and produce consistent results across a range of test conditions. The evaluation demonstrates that the implemented system satisfies the core functional requirements defined in Chapter 3.

6.3 System-Level Performance Evaluation

While the previous section validated individual functional components, this section evaluates the performance of the complete system pipeline. The objective is to

assess how effectively the integrated system operates under realistic conditions, considering execution efficiency, motion quality, and overall system behaviour.

6.3.1 End-to-End Execution Results

The system is evaluated using the test scenarios defined in Section 6.1.2, covering variations in workpiece geometry, orientation, and spatial placement. Across all evaluated scenarios, the system successfully generated and executed complete robot programs without requiring manual intervention.
The results demonstrate that the planning pipeline operates cohesively, with each module providing valid input to subsequent stages. Orientation determination, grip planning, regrip handling, and trajectory generation function together to produce a complete and executable workflow. The system consistently adapts to varying initial conditions, confirming its ability to handle non-deterministic workpiece placement.

6.3.2 Execution Time Analysis

The total execution time is divided into planning time and motion execution time. Planning time includes all computations required to generate a valid motion sequence, while execution time corresponds to the robot motion within the environment.
The evaluation shows that planning time remains significantly lower than execution time, indicating that the system is suitable for near real-time operation. Even in scenarios requiring regrip operations, the additional computational overhead remains limited due to the use of structured decision strategies and constrained search spaces.

6.3.3 Trajectory Quality and Feasibility

Trajectory quality is assessed based on smoothness, continuity, and kinematic feasibility. The generated trajectories exhibit continuous motion without abrupt changes in direction or velocity. The use of interpolated waypoints and smooth motion profiles ensures stable execution.
Workspace constraints and singularity avoidance mechanisms prevent the generation of infeasible paths. When target poses approach the limits of the workspace, the system adapts the trajectory to remain within valid bounds. This confirms that the generated motions are both feasible and reliable for execution.

6.3.4 Comparison with Manual Programming

To evaluate the practical benefits of the developed system, a comparison is performed between conventional manual programming using drag&bot function blocks and the automated approach proposed in this work.

In the manual approach, the operator defines individual motion steps, configures parameters, and ensures correct sequencing of operations. This process is time-intensive and depends strongly on operator expertise. In contrast, the developed system generates the complete robot program automatically based on geometric input and configuration data.
Table 6.1 summarises the comparison between the two approaches.

Aspect	**Manual Programming (drag&bot blocks)**	**Automated System**
Programming effort	High (manual definition of motion steps)	Minimal (automatic program generation)
Reprogramming after changes	Required for each modification	Not required (automatic recalculation)
Adaptation to position/orientation changes	Manual adjustment of waypoints required	Automatically handled by planning pipeline
Adaptation time per change	5–10 minutes	10–30 seconds
Error likelihood	Higher due to manual input	Reduced through validation and constraints
Repeatability	Dependent on operator consistency	High due to deterministic execution
Handling of complex operations (regrip, flip)	Requires manual planning and sequencing	Automatically handled within system pipeline
Flexibility for new workpieces	Requires reprogramming	Handled through input and configuration changes

Table 6.1: Comparison between manual programming and automated system under varying conditions.

The comparison demonstrates that the proposed system significantly reduces programming effort while improving consistency and reliability. By eliminating manual waypoint definition and automating decision-making processes, the system achieves the primary objective of reducing operator workload and increasing process efficiency.

6.4 Robustness and Failure Analysis

In addition to functional correctness and performance, the robustness of the system is evaluated by analysing its behaviour under challenging and failure-prone conditions. This assessment is closely related to the Failure Mode and Effects Analysis (FMEA) presented in Section 5.11.

6.4.1 Failure Case Evaluation

The system is tested under scenarios designed to trigger potential failure modes, including invalid workpiece placement, infeasible grip configurations, and workspace boundary violations. In each case, the system successfully detects the issue before or during execution.
For example, configurations with insufficient vacuum holding force are rejected during grip planning, while unreachable poses are identified during trajectory generation. These validations prevent the execution of unsafe or infeasible motions.

6.4.2 System Response to Errors

When a failure condition is detected, the system responds by halting execution and notifying the operator through the human interaction interface. This behaviour ensures that errors are clearly communicated and prevents the continuation of unsafe operations.
The runtime error monitoring mechanism further enhances robustness by detecting execution-level failures and providing immediate feedback. This layered approach to error handling ensures that both planning and execution errors are addressed.

6.4.3 Limitations of the System

Despite its robustness, the system exhibits certain limitations. The evaluation is conducted within a virtual environment, and therefore does not capture all real-world uncertainties such as sensor noise, calibration errors, or mechanical tolerances.
Additionally, the absence of full self-collision detection limits the ability to guarantee collision-free operation in highly constrained scenarios. These limitations highlight areas for future improvement and potential extension of the system.

6.5 Discussion

The evaluation results demonstrate that the developed system provides a structured and automated solution for press brake bending. Compared to the approaches discussed in Chapter 2, the system eliminates the need for manual programming and predefined sequences, enabling adaptive behaviour based on real-time geometric input.
The integration of orientation determination, grip planning, and regrip handling allows the system to operate under a wide range of conditions. This represents a significant improvement over conventional methods, which typically rely on fixed configurations and require manual intervention for complex tasks.
The system also achieves a high degree of modularity, as demonstrated by the layered architecture presented in Chapter 5. Each module performs a well-defined function and contributes to the overall pipeline, supporting maintainability and extensibility.

However, the evaluation also reveals limitations. The reliance on a virtual simulation environment means that the results do not fully reflect real-world operating conditions. Furthermore, certain aspects, such as advanced collision detection and dynamic force feedback, are not included in the current implementation.
Overall, the system addresses key gaps identified in the state of the art by providing an automated, adaptable, and modular solution. At the same time, the identified limitations provide a clear direction for future development.

7 Conclusion and Future Work

7.1 Conclusion

This thesis presented the development and evaluation of an automated robotic system for press brake sheet metal bending. The objective was to reduce manual programming effort while improving adaptability and execution reliability in industrial bending operations.

The analysis of the state of the art in Chapter 2 identified key limitations in existing approaches, including a strong dependence on manual programming, limited adaptability to changing workpiece conditions, and a lack of integrated planning frameworks. Based on these observations, a set of functional and non-functional requirements was defined in Chapter 3, focusing on automation, flexibility, and system robustness.

In Chapter 4, multiple solution concepts were evaluated, leading to the selection of Concept B as the most suitable approach. This concept introduced a structured planning pipeline that integrates orientation determination, grip planning, regrip handling, and trajectory generation into a unified system.

Chapter 5 described the complete implementation of the selected concept. The system was realised as a modular software architecture with clearly defined layers, enabling separation of concerns and maintainability. Key functionalities such as constraint-based grip planning, automatic regrip determination, and structured trajectory generation were implemented to support fully automated operation.

The evaluation presented in Chapter 6 demonstrated that the system successfully fulfils the defined requirements. Functional validation confirmed that each component of the planning pipeline operates correctly under varying conditions. System-level evaluation showed that the integrated solution produces consistent and feasible robot programs without manual intervention.

A comparison with manual programming approaches highlighted a significant reduction in programming effort and adaptation time. Tasks that traditionally require manual adjustment and reprogramming can be handled automatically by the system, enabling faster and more consistent operation. This demonstrates that the primary objective of reducing operator dependency and improving efficiency has been achieved.

Overall, the developed system provides a systematic and adaptable solution for robotic press brake bending. By combining automated decision-making with structured validation, the approach addresses key limitations of existing methods and contributes to the advancement of flexible robotic manufacturing systems.

7.2 Contributions

The main contributions of this thesis can be summarised as follows:

- Development of a fully automated planning pipeline for press brake bending, integrating orientation determination, grip planning, regrip handling, and trajectory generation
- Introduction of a constraint-based grip planning method that ensures both geometric feasibility and sufficient holding force
- Design of an adaptive regrip strategy that determines required reorientation dynamically without predefined sequences
- Implementation of a structured trajectory generation approach that ensures smooth and feasible robot motion
- Integration of validation mechanisms for safety and feasibility, including pre-execution checks and runtime monitoring
- Demonstration of reduced programming effort and improved adaptability through comparison with manual programming approaches

These contributions collectively address the challenges identified in the state of the art and provide a foundation for further development of automated robotic bending systems.

7.3 Limitations

Despite the successful implementation and evaluation of the system, several limitations remain.
The evaluation was conducted entirely within a virtual simulation environment. While this enables controlled testing, it does not fully capture real-world factors such as sensor noise, calibration inaccuracies, and mechanical tolerances.
In addition, the system does not include comprehensive collision detection or advanced dynamic modelling of contact interactions. These aspects are important for ensuring safe operation in more complex or constrained environments.
Furthermore, the current implementation focuses on a specific class of sheet metal geometries and bending operations. More complex geometries or multi-step industrial processes may require additional extensions to the planning pipeline.

7.4 Future Work

Based on the identified limitations, several directions for future work can be proposed.

Although the developed system achieves a functional and structured approach to robotic sheet handling and bending, it is inherently constrained by a series of simplifying assumptions introduced during the design phase. These constraints, while beneficial for ensuring stability and predictability, limit the system's applicability to more complex and realistic industrial scenarios. Consequently, several avenues for future work emerge, aimed at improving flexibility, intelligence, and robustness.

A key limitation of the current system lies in its strict assumption of rectangular sheet geometries aligned with the global coordinate frame. In practical manufacturing environments, components often exhibit irregular shapes and arbitrary orientations. Future work should therefore focus on generalizing the geometric representation to support non-rectangular profiles, including polygonal and free-form geometries. Additionally, enabling the system to operate independently of fixed axis alignment would significantly enhance its adaptability. This would require a more advanced geometric abstraction layer capable of handling arbitrary edge definitions and diverse feature types, including non-circular cutouts.

The bending capabilities of the system also present a significant opportunity for advancement. The restriction to single, 90-degree bends per edge simplifies the planning process but does not reflect the complexity of real-world fabrication tasks. Future developments should introduce support for variable bend angles and multiple bends along a single edge. More importantly, this necessitates the formulation of an intelligent bend sequence planning framework capable of optimizing operation order, minimizing regrip requirements, and ensuring process feasibility. Such a framework could leverage constraint-based optimization or heuristic search methods to generate efficient and manufacturable bending strategies.

From a robotics perspective, motion planning remains an area with considerable scope for improvement. The current system demonstrates reduced efficiency when handling certain orientations, particularly those involving significant rotational transformations. Future work should integrate advanced motion planning techniques that ensure collision-free, time-optimal trajectories. Furthermore, introducing adaptive calibration mechanisms would allow the system to remain functional even when the robot base or surrounding fixtures are repositioned, thereby reducing dependency on manual reconfiguration. The incorporation of learning-based methods could further enhance performance by enabling the system to refine its motion strategies through repeated execution.

The gripping subsystem, currently limited to vacuum-based handling, represents another constraint on system versatility. While effective for flat and uniform surfaces, vacuum gripping is not universally applicable. Expanding the system to support multiple gripping modalities, including mechanical and magnetic solutions, would broaden its operational scope. In addition, the integration of sensor feedback, such as force or slip detection, would enable real-time validation of grasp stability and improve overall reliability.

Regrip operations in the present implementation are restricted to discrete rotational

increments, which limits the flexibility of part orientation. Future work should explore continuous orientation control and develop more sophisticated regrip planning strategies aimed at minimizing intermediate handling steps. Streamlining opposite-side operations into a unified workflow, rather than relying on separate scripts, would further improve system efficiency and maintainability.

Safety considerations represent a critical gap in the current system. The absence of collision detection and trajectory validation poses a significant limitation for deployment in real-world environments. Future enhancements should prioritize the integration of real-time collision avoidance mechanisms and safe motion planning algorithms. Expanding the scope of the safety monitoring system to account for dynamic interactions between the robot, workpieces, and surrounding infrastructure would also be essential.

Finally, the system's reliance on fixed object identifiers and predefined configurations reduces its robustness in dynamic environments. Future work should investigate the integration of perception-driven approaches, such as vision-based object recognition and tracking, to enable more flexible and autonomous operation. Additionally, incorporating error detection and recovery strategies would allow the system to handle unexpected deviations more effectively, thereby improving overall resilience.

In conclusion, future developments should aim to transition the system from a rigid, rule-based framework toward a more adaptive and intelligent architecture. By addressing limitations in geometry handling, bending flexibility, motion planning, gripping strategies, regrip operations, safety, and system adaptability, the proposed solution can evolve into a more comprehensive and industrially viable robotic automation platform.

7.5 Final Remarks

This thesis demonstrates that automated planning and execution of robotic press brake bending is both feasible and beneficial. By reducing manual programming effort and improving adaptability, the developed system contributes to more efficient and flexible manufacturing processes.

The presented approach provides a structured foundation for future research and development in robotic bending and related automation tasks.

Bibliography

[1] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998. DOI: 10.1016/S0278-6125(98)80002-2

[2] S. Aomura and A. Koguchi, "Optimized bending sequences of sheet metal bending by robot," *Robotics and Computer-Integrated Manufacturing*, vol. 18, no. 1, pp. 29–39, 2002. DOI: 10.1016/S0736-5845(01)00031-X

[3] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 28, no. 2, pp. 87–94, 2012. DOI: 10.1016/j.rcim.2011.08.004

[4] P. Neto, "Off-line programming and simulation from cad drawings: Robot-assisted sheet metal bending," in *Proceedings of the 39th Annual Conference of the IEEE Industrial Electronics Society (IECON)*, Department of Mechanical Engineering, CEMUC, Vienna, Austria, 2013, pp. 4235–4240. DOI: 10.1109/IECON.2013.6699827

[5] Universal Robots A/S, *UR5e Technical Specifications*, Odense, Denmark, 2026. Accessed: Apr. 20, 2026. [Online]. Available: https://www.universal-robots.com/manuals/EN/HTML/SW5_24/Content/prod-usr-man/complianceUR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm

[6] *Robots and robotic devices — vocabulary*, International Organization for Standardization, 2012.

[7] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005, ISBN: 978-0-13-123629-5.

[8] M. R. Pedersen et al., "Robot skills for manufacturing: From concept to industrial deployment," *Robotics and Computer-Integrated Manufacturing*, vol. 37, pp. 282–291, 2016. DOI: 10.1016/j.rcim.2015.04.002

[9] G. Pahl, W. Beitz, J. Feldhusen, and K. H. Grote, *Engineering Design: A Systematic Approach*, 3rd ed. London: Springer, 2007. DOI: 10.1007/978-1-84628-319-2

[10] Acieta, *Robotic press brake, welding & palletizing cell for cotterman*, 2018. Accessed: Mar. 27, 2026. [Online]. Available: https://www.acieta.com/automation-application/robotically-integrated-bending-solutions-ribs/

[11] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, 1st ed. London: Springer, 2009, ISBN: 978-1-84628-641-4.

[12] RoboDK, *Robodk: Simulation and offline programming software for industrial robots*, 2024. Accessed: Mar. 27, 2026. [Online]. Available: https://robodk.com

[13] F. Tao, M. Zhang, and A. Nee, *Digital Twin Driven Smart Manufacturing*. Academic Press, 2019, ISBN: 978-0-12-817630-6. DOI: 10.1016/C2018-0-02206-9

[14] T. Lozano-Pérez, "Robot programming," *Proceedings of the IEEE*, vol. 71, no. 7, pp. 821–841, 1983. DOI: 10.1109/PROC.1983.12678

[15] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-driven grasp synthesis – a survey," *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, 2014. DOI: 10.1109/TRO.2013.2289018

[16] W. Wan and K. Harada, "Regrasp planning using 10,000s of grasps," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1929–1936.

[17] F. Gabriel, M. Fahning, J. Meiners, F. Dietrich, and K. Dröder, "Modeling of vacuum grippers for the design of energy efficient vacuum-based handling processes," *Production Engineering*, vol. 14, pp. 545–554, 2020. DOI: 10.1007/s11740-020-00990-9

[18] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Springer, 2008.

[19] drag&bot GmbH. "Anyrobot interface documentation." Ulm, Germany, Accessed: Mar. 2, 2026. [Online]. Available: https://docs.dragandbot.com/anyrobot-interface

[20] K. Thramboulidis, "Model-driven engineering for industrial automation systems: A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 1893–1903, 2013. DOI: 10.1109/TII.2013.2251233

[21] M. Bär, G. Reinhart, and J. Schilp, "Virtual commissioning of robotic systems: A review and outlook," *Procedia CIRP*, vol. 72, pp. 129–134, 2018.

[22] M. Quigley et al., "ROS: an Open-Source Robot Operating System," in *Proceedings of the ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[23] V. Pimentel and B. G. Nickerson, "Communicating and displaying real-time data with websocket," *IEEE Internet Computing*, vol. 16, no. 4, pp. 45–53, 2012. DOI: 10.1109/MIC.2012.64

[24] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: Ros for non-ros users," in *Robotics Research*, Springer, 2017, pp. 493–504.

[25] F. Tao, H. Zhang, A. Liu, and A. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019. DOI: 10.1109/TII.2018.2873186

[26] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022. DOI: 10.1126/scirobotics.abm6074

[27] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 9781107156302.

[28] A. Bicchi and V. Kumar, "Robotic grasping and contact: A review," *ICRA*, 2000.

[29] Schmalz GmbH, *Vacuum handling systems: Design principles and force calculation*, Technical reference for vacuum gripping force models, 2020.

[30] K. Shoemake, "Animating rotation with quaternion curves," in *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, ACM, 1985, pp. 245–254. DOI: 10.1145/325334.325242

[31] T. Flash and N. Hogan, "The coordination of arm movements: An experimentally confirmed mathematical model," *The Journal of Neuroscience*, vol. 5, no. 7, pp. 1688–1703, 1985. DOI: 10.1523/JNEUROSCI.05-07-01688.1985

[32] A. Gasparetto, P. Boscariol, A. Lanzutti, and R. Vidoni, "Trajectory planning in robotics," *Mathematics in Computer Science*, vol. 6, no. 3, pp. 269–279, 2012. DOI: 10.1007/s11786-012-0123-8